

Generative AI: Foundations, Applications, and Emerging Trends



Abhishek Patange, Ph.D.

Data Science and AI Specialist

ABB Global Industries and Services Pvt. Ltd.

Bengaluru

Different approaches to automation

Different approaches to automation



Mechanical

Electronic / Electrical

Pneumatic

Hydraulic

Chemical



Thermal

Optical

Acoustical

Software

👉 Are we choosing the right approach — or just the latest one?

that's exactly where we are with AI today!

Rule-based → Expert systems → ML → GenAI / LLMs → AI Agents → Agentic AI

Today, there's a strong pull toward Agentic AI — systems that can plan, decide, and act independently.

It's powerful, no doubt.

But before we jump in and “solutionalize” everything with it, we should step back and ask:

- Do we need autonomy, or just well-defined decision logic?
- Is the problem well-defined, or are we adding complexity to mask ambiguity?
- Are we introducing probabilistic behavior where determinism matters?
- What's the trade-off in cost, observability, and control?
- Can a deterministic or human-in-the-loop system actually perform better?
- Are we optimizing for impact or just innovation optics?

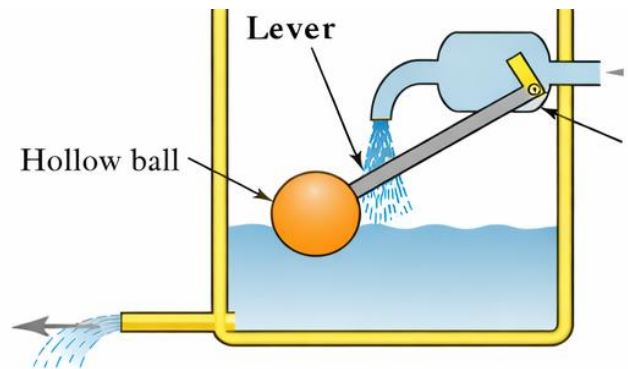
- 💡 Not every problem needs an agent.
- 💡 Not every workflow needs intelligence.
- 💡 And not every advancement needs to be adopted immediately.

The real skill today isn't just building with the latest paradigm — it's choosing when not to use it.

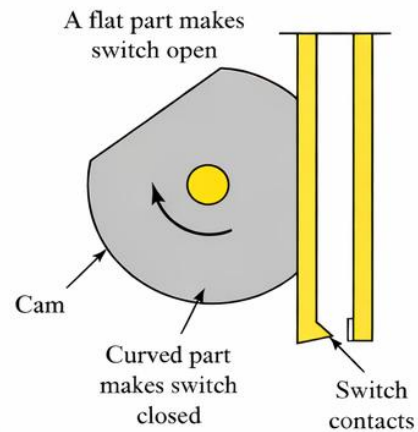
We need to move forward, but with intent — not inertia!!

Classic approach

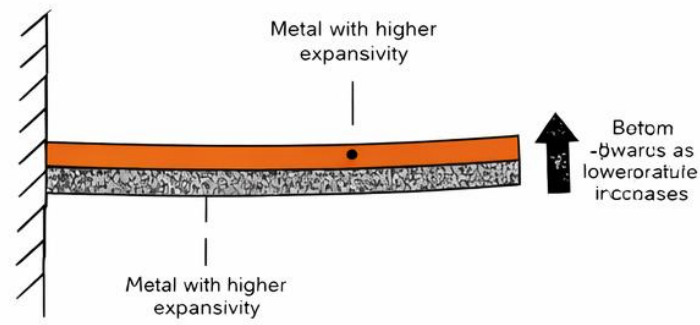
Classic approach



Float Valve



Cam Operated Switch



Bi-metallic Strip

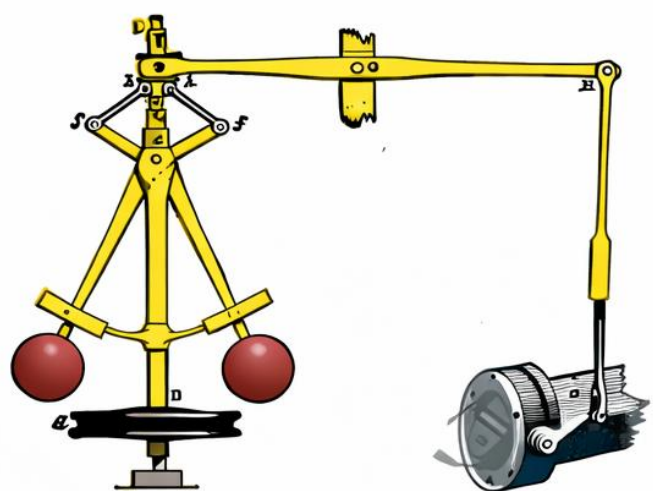
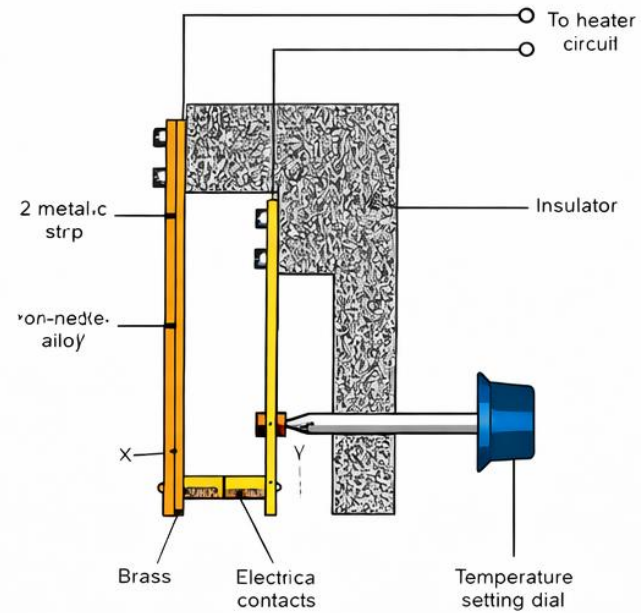
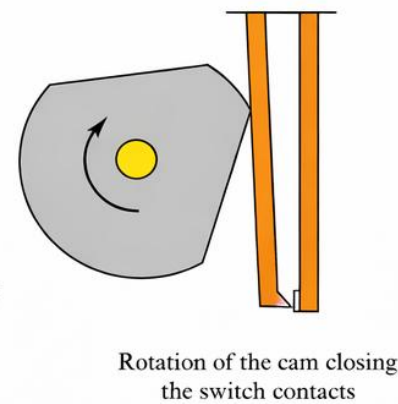


FIG. 4—Governor and Throttle-Valve.

Watt's Governor



Thermostat

Rule based

vs.

Machine Learning

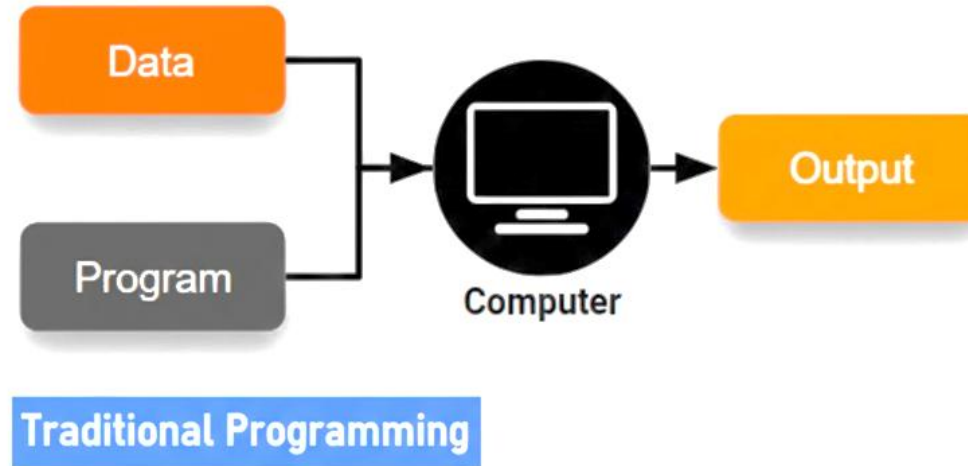
Rule based vs. Machine Learning

When **relationship between input and output is explicit**, rules (program) can be manually written

Dataset	
A	B
2	3
4	2
5	3
1	1

Program

$C = A + B$



Output		
A	B	C
2	3	5
4	2	6
5	3	8
1	1	2

When relationship between input and output is **not explicitly known**, computer learns the rules (program/patterns) from dataset itself

Dataset

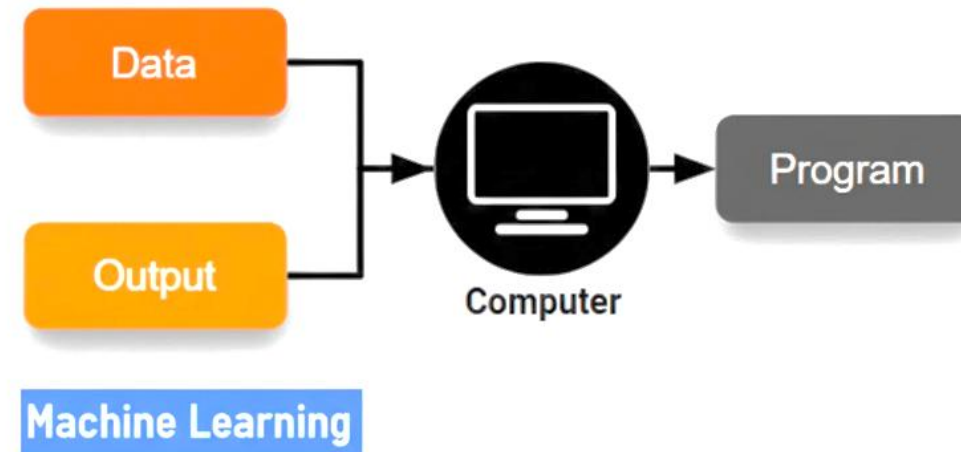
In form of Input Features (X):

- Speed (mph)
- Acceleration (m/s^2)
- Distraction Level (1–10)

Output

Label / driving state / class (y):

- Normal Driving
- Aggressive Driving
- Distracted Driving



```
if speed <= 68:
    result = "Normal Driving (50.0)"
else: # speed > 68
    if acceleration <= 4.5:
        result = "Aggressive Driving (25.0)"
    else: # acceleration > 4.5
        if distraction_level <= 6:
            result = "Aggressive Driving (5.0)"
        else: # distraction_level > 6
            result = "Distracted Driving (20.0)"
```

Program

Artificial Intelligence

Artificial Intelligence

AI is the ability of machines to mimic human intelligence. Whenever a system can think, learn, decide, sense, or act like a human.

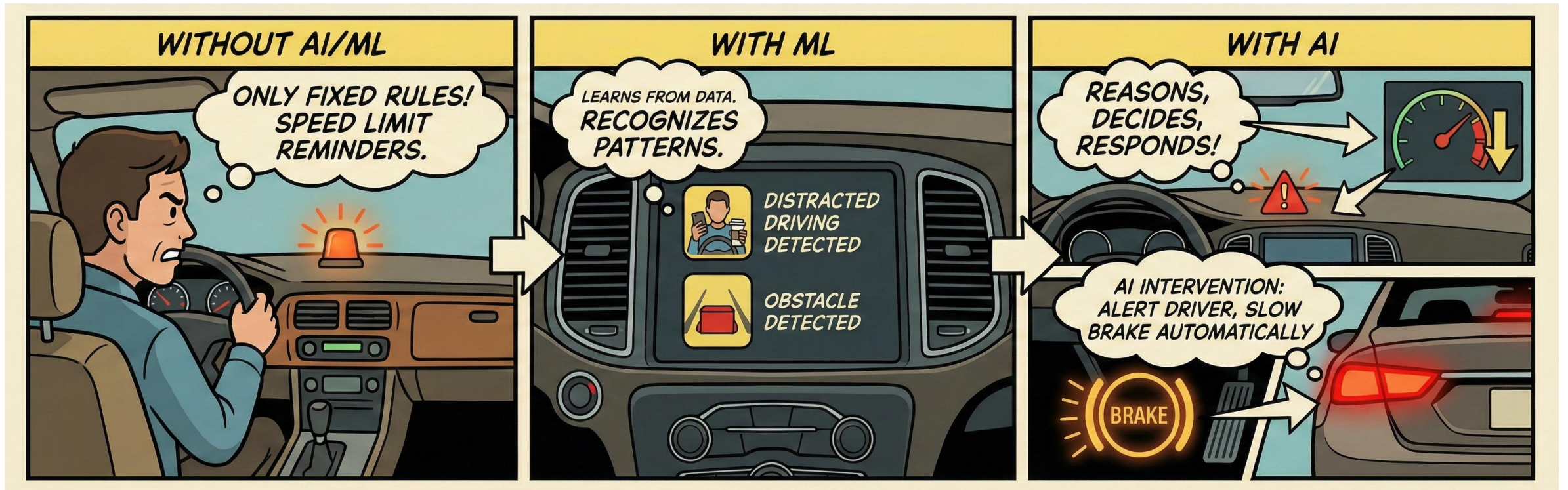
Let's break down this definition in steps

AI is the **GOAL: (What we want)**

Can the machine *act intelligently? make decisions? Understands?*

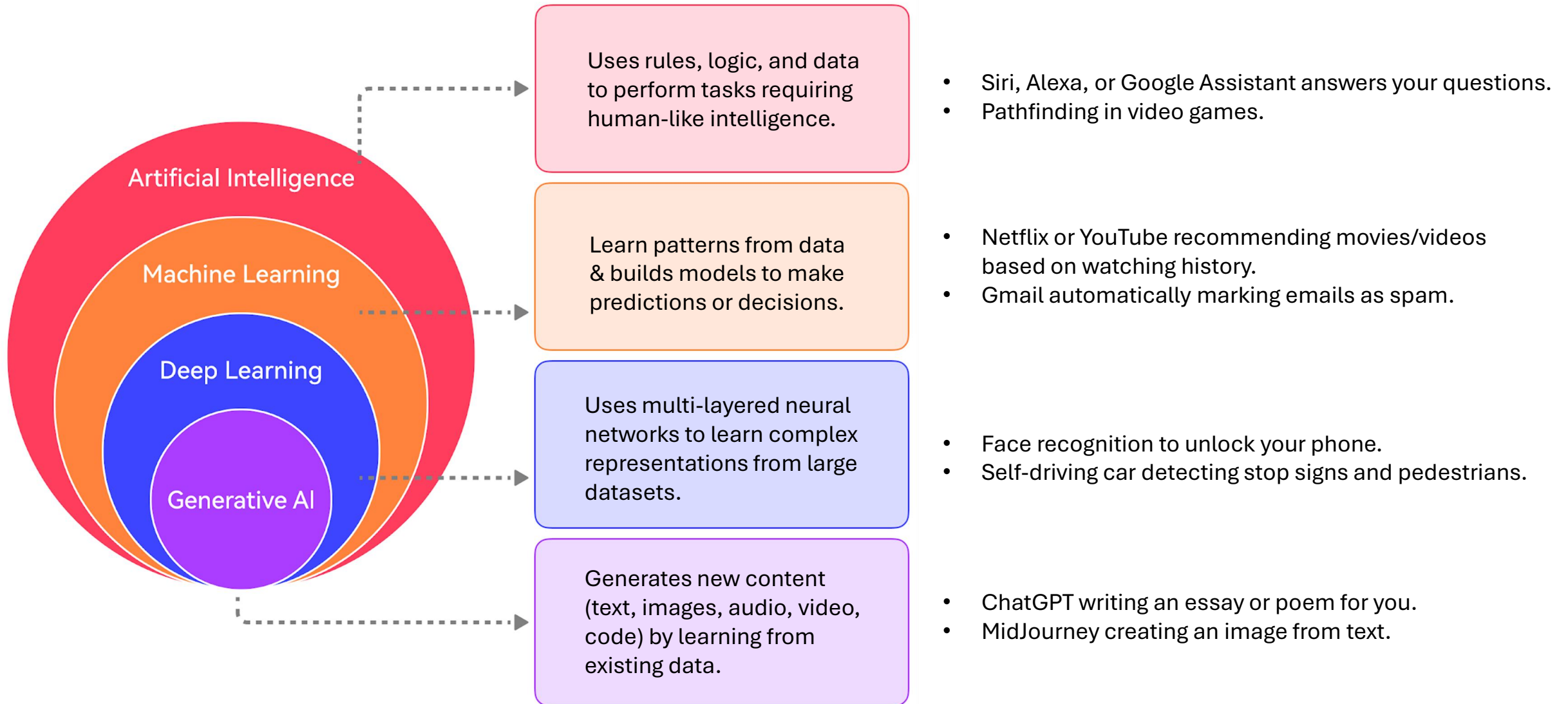
ML is the **METHOD: (How we get there)**

Learns patterns from data instead of being programmed



Generative AI

The AI Family: From Intelligence to Creativity



Understanding term “Generative”

It was all started with **Google predictive search**. It trains a LLM on billions of search queries made by users over the years, which then tries to predict the next word in your own search query.



Q what is definition of

Q what is definition of **done in agile**

Q what is definition of **noun**

Q what is definition of **love**

Q what is definition of **motion**

Q what is definition of **computer**

Q what is definition of **cell**

Q what is definition of **success**

Q what is definition of **quality**

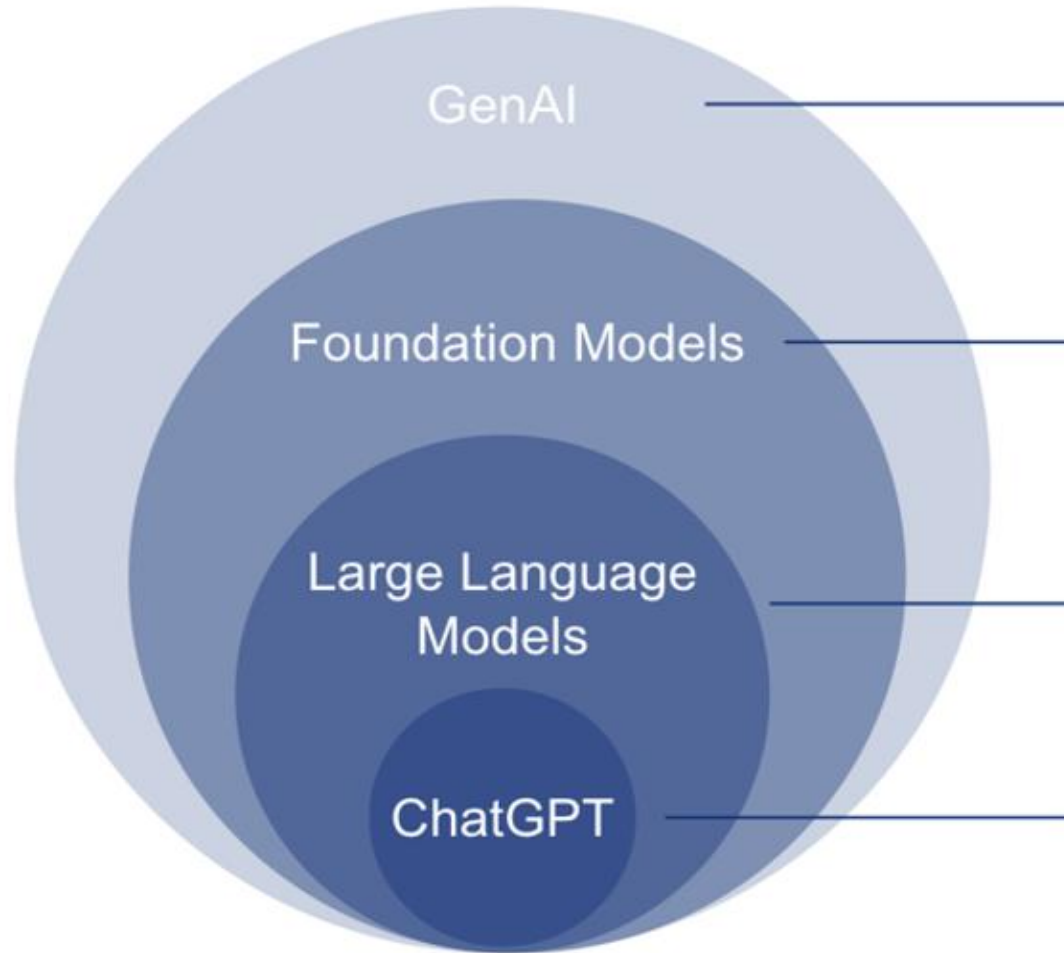
Q what is definition of **science**

Q what is definition of **area**

Feature	Predictive Search (Old School)	Generative AI (Next-Gen)
Goal	Completes your query by predicting the next word	Creates entirely new content (text, images, audio, video, code)
Training Data	Billions of past search queries	Massive datasets (books, articles, images, code etc.)
Output Type	Suggestions / autocomplete	Original, coherent, creative content
Example	You type “Best hotels in...” → Suggests “Pune,” “Mumbai”	You ask “Write a 500-word essay on smart irrigation in villages” → Produces a full essay
Technology Base	Statistical + simple machine learning	Large Language Models (LLMs), Transformers, Generative Models
Analogy	A friend who finishes your sentence	A friend who writes an entire book, paints, or composes a song

Large language models

Generative AI to ChatGPT: How it all connects?



Type of AI that creates new content like text, images, or music using a lot of data. Like making a poem, image, or song using just a few words you give it.

These are big, general-purpose NN models trained on huge data, and they can understand multimodal information, answer questions, or describe images.

A special foundation model that focuses only on language (text). It learns from reading lots of text, so it can write essays, chat, translate, etc.

This is a chat app built using an LLM. You type a question, and it gives a smart answer — like you're chatting with a human.

Chat: natural language system

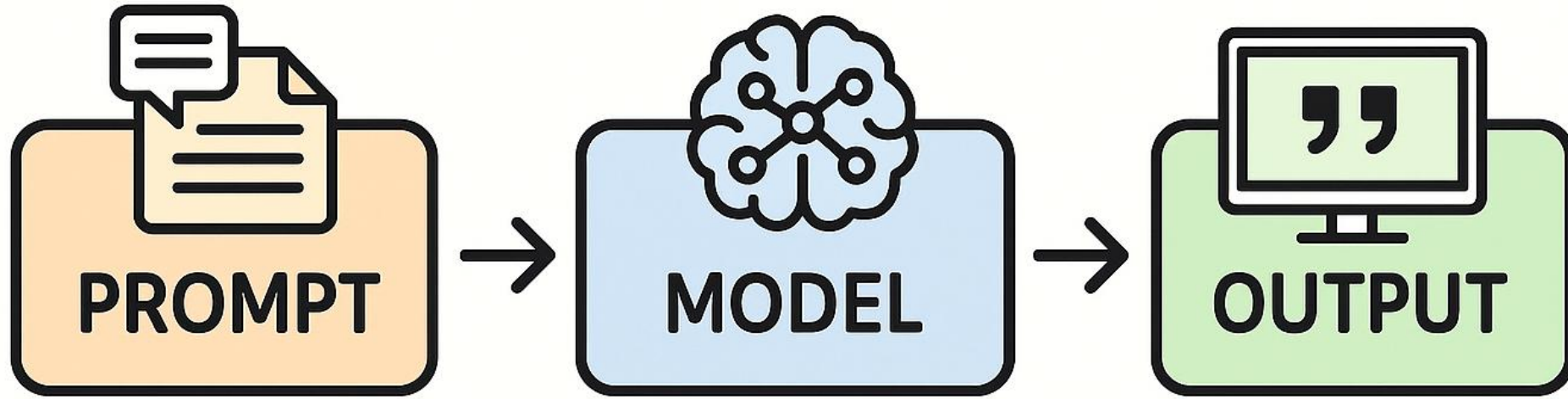
Explained
Today

G: Generatively – Designed to model the creation of text

P: Pretrained – Trained on lots of naturally occurring data

T: Transformer – A kind of neural network architecture

Building blocks of LLMs



The instruction or question given to the LLM. It provides context, task description, or data for the model to process.

The Large Language Model processes the prompt using patterns learned during training and predicts the most relevant sequence of words.

The response produced by the model based on the prompt and its learned knowledge. It can be text, code, summaries, translations, etc.

Capabilities of LLMs

Generative AI / LLM capabilities mainly fall into three groups:

Understanding	
• Natural Language Understanding – interpret user intent and context • Information Extraction – identify structured data from text • Translation – convert text between languages • Conversation Comprehension – maintain context across dialogue	• Understanding “What are the benefits of renewable energy?” • Extracting names, dates, and locations from a document • Translating English → Spanish • Answering follow-up questions in a conversation
Generation	
• Text Generation – create articles, emails, or reports • Code Generation – produce programming code from instructions • Creative Writing – generate stories, poems, scripts • Content Creation – marketing content or product descriptions • Multimodal Generation – generate images or other media from text	• Writing a blog post about climate change • Generating a Python function to sort a list • Creating a short science-fiction story • Writing product descriptions for an e-commerce site • Generating images from text prompts
Reasoning	
• Problem Solving – analyze and solve mathematical or logical problems • Logical Reasoning – derive conclusions from given information • Question Answering – answer factual or conceptual questions • Summarization – condense long content into key points	• Solving 2x + 5 = 15 step-by-step • Determining logical outcomes in a scenario or puzzle • Answering “What is photosynthesis?” • Summarizing a 20-page research paper into key insights

If LLMs are so powerful, why do we need something more?

Limitations of LLMs

Limitation	Explanation & Example
No Real-World Actions	Generate responses but cannot directly perform actions in external systems. Example: Cannot send emails or update a database.
No Direct Access to External Data	Knowledge is limited to training data unless connected to external tools. Example: Cannot fetch research papers from Google Scholar.
Limited Memory	Have limited context windows and cannot maintain long-term memory. Example: Cannot remember previous conversations across sessions.
Requires Human Prompts	Respond to instructions rather than initiating tasks independently. Example: Needs a user to request each step of a task.
Limited Task Execution	Can suggest solutions but cannot execute complex workflows. Example: Can suggest a marketing strategy but cannot analyze company sales data automatically.
Hallucination Risk	May generate incorrect or fabricated information confidently. Example: Providing incorrect references or facts.

LLMs are excellent at **thinking and generating language**, but they are not designed to **act or execute tasks**.

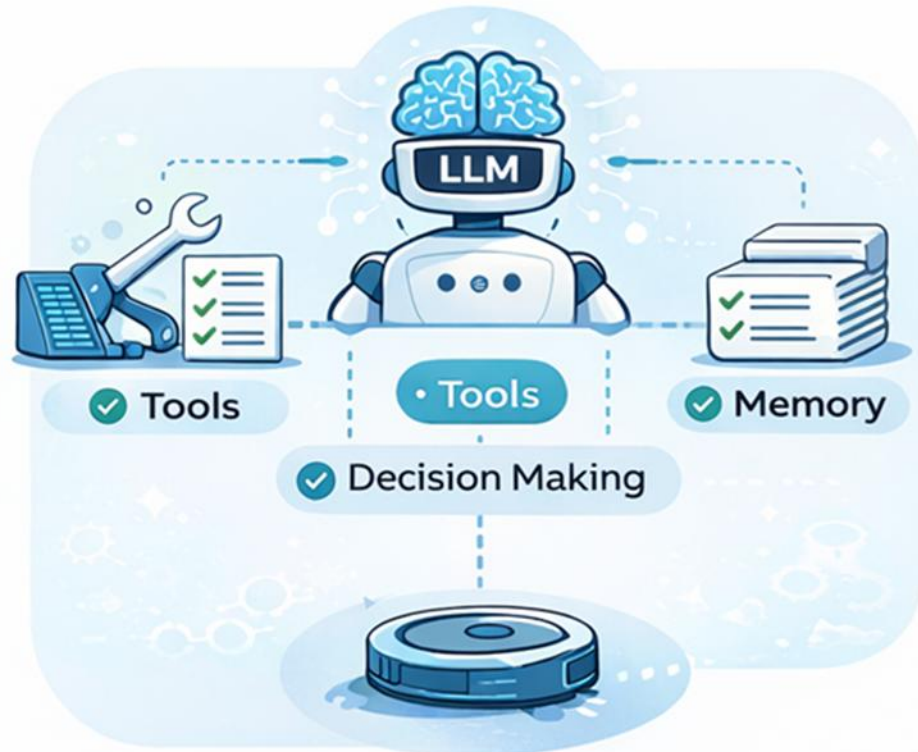
AI Agents & Agentic AI

AI Agents & Agentic AI

AI Agents

An **AI Agent** is a system that uses a **Large Language Model (LLM)** together with **tools**, **memory**, and **decision logic** to perform actions and **achieve a goal autonomously**.

AI Agent = LLM + Tools + Memory + Decision Making



Agentic AI

Agentic AI refers to AI systems that can **autonomously plan, coordinate, and execute complex tasks** by managing one or multiple AI agents to achieve a goal.

Goal-driven autonomy and multi-step decision making.



AI Agents

AI Agents

An **AI Agent** is a system that uses a **Large Language Model (LLM)** together with tools, memory, and decision logic to **perform actions and achieve a goal autonomously**.

AI Agent = LLM + Tools + Memory + Decision Making

The LLM provides reasoning ability, while the agent framework allows it to interact with external systems.

Building blocks of AI agents

Component	Description	Example
LLM (Reasoning Engine)	Understands instructions and decides what to do.	Models used in systems like ChatGPT
Tools / APIs	Allows the agent to interact with external systems or data sources.	Web search, databases, APIs
Memory	Stores previous interactions and context for better decision-making.	Remembering previous user queries
Planning / Decision Logic	Determines the sequence of actions required to achieve a goal.	Breaking a complex task into steps

LLMs are excellent at **thinking and generating language**, but they are not designed to **act or execute tasks**.

AI Agents example

User Goal: “Analyze our sales data and create a marketing report.”

Agent Workflow

- 1. Understand the request using the LLM
- 2. Retrieve sales data from a database
- 3. Analyze trends using analytics tools
- 4. Generate a report
- 5. Present recommendations

Agent Frameworks

- **LangChain** – Developed by **Harrison Chase**, it is a framework for building LLM-powered applications by connecting language models with tools, data sources, and memory to create AI agents and workflows.
- **AutoGPT** – Created by **Toran Bruce Richards**, it is an open-source autonomous agent system that uses LLMs to break down goals into tasks and execute them with minimal human intervention.

System	Capability
LLM	Understands and generates language
AI Agent	Understands, plans, and performs actions

While a single AI agent can perform tasks using tools, more advanced systems coordinate multiple agents to achieve complex goals. This leads to the concept of **Agentic AI**.

Agentic AI

Agentic AI

Agentic AI refers to AI systems that can **autonomously plan, coordinate, and execute complex tasks by managing one or multiple AI agents to achieve a goal.**

Unlike a single AI agent, Agentic AI focuses on **goal-driven autonomy and multi-step decision making.**

Characteristic	Description	Example
Autonomous Planning	System breaks a goal into multiple tasks automatically.	Planning steps to conduct market research
Multi-Agent Collaboration	Multiple agents work together to solve a problem.	Research agent + analysis agent + reporting agent
Dynamic Decision Making	System adapts its plan based on results or feedback.	Changing strategy if data analysis fails
Tool Integration	Agents interact with external systems and APIs.	Using databases, web search, analytics tools

- Goal:** “Create a market analysis report.”
- **Research Agent** collects market data
 - **Analysis Agent** analyzes trends and statistics
 - **Content Agent** writes the report
 - **Review Agent** checks accuracy and improves the output

System	Capability
AI Agent	Performs tasks using tools based on a given goal
Agentic AI	Coordinates multiple agents to autonomously plan and complete complex goals

Agentic AI represents the next evolution of AI systems, moving from simple task execution toward fully autonomous goal-driven systems.

**Take one use case and screen through
TP, ML, LLM, AI Agent and Agentic AI**

Comparing RB, ML and LLMs considering floor cleaning robot

1. Rule-Based Robot

Robot follows fixed programmed rules

How it works

- Move forward
- If obstacle → turn
- Vacuum while moving
- Stop when battery is low

Behavior: “Move straight. If wall detected, turn right. Continue cleaning until battery reaches 5%.”

Characteristics

- Deterministic
- No learning
- Cheap and predictable
- Struggles with complex environments

2. Machine Learning (ML) Robot

Robot uses trained ML models to recognize patterns and optimize cleaning

How it works

- Collect sensor/camera data
- ML model identifies dirtier regions
- Predict efficient cleaning paths
- Adjust suction based on floor type

Behavior: “This area usually accumulates more dust near the dining table. Increase cleaning frequency here.”

Characteristics

- Learns from data
- Better efficiency
- Can classify carpet, tile, wet/dry areas
- Still task-specific

3. LLM-Based Robot

Uses an LLM to reason in natural-language

How it works

- Converts sensor info into structured descriptions
 - Interprets the situation/ Gives high-level decisions
- Input to the LLM:** Room layout / Dirt levels / Obstacle locations / Battery status / User instructions

Behavior: “The living room is already clean, but the kitchen has sticky spills. Clean the kitchen first and avoid the sleeping dog.”

Characteristics

- Context-aware reasoning
- Understands user instructions
- Flexible decision-making
- May hallucinate or make inefficient plans

4. AI Agent Robot

Autonomously observes, plans, acts, and adapts

Agent Loop

Observe → Think → Plan → Act → Evaluate → Repeat

How it works

- Understand the cleaning goal
- Analyze the environment continuously
- Decide priorities dynamically / Execute actions
- Monitor outcomes and recover from failures

Behavior: “The kitchen has the highest dirt level, but battery is only 18%. First recharge, then clean the kitchen and hallway.”

Capability: Goal-driven, not just instruction following.

Characteristics

- Autonomous decision-making
- Dynamic replanning / Memory and feedback loops
- Handles unexpected situations

5. Multi-Agent / Agentic AI System

Multiple specialized AI agents collaborate together

Example Agents

Mapping Agent → Builds room map

Planning Agent → Creates cleaning strategy

Execution Agent → Controls movement and cleaning

Monitoring Agent → Detects failures or missed spots

Resource Agent → Manages battery, water, and charging

Collaborative Workflow

Mapping → Planning → Execution → Monitoring → Optimization

Example behavior

Mapping Agent: “Kitchen and hallway are dirty.”

Planning Agent: “Clean hallway first to minimize travel.”

Resource Agent: “Battery sufficient for 25 more minutes.”

Execution Agent: “Starting hallway cleaning now.”

Characteristics

Modular intelligence

Parallel reasoning

More scalable and robust

Suitable for complex real-world automation

Evolution of Intelligent Systems: From Traditional Programming to Agentic AI

Goal: Apply for a Software Engineer job

Step	Traditional Programming	Machine Learning	LLM / Chatbot	AI Agent	Agentic AI
1. Understand Goal	Goal must be explicitly hardcoded. Example: "Apply only to jobs from Company X."	ML model classifies whether a job is relevant based on training data.	User says: "Apply for backend engineering jobs in Berlin." LLM understands the request in natural language.	Agent interprets goals, priorities, constraints, deadlines, salary expectations, and preferred technologies.	Multiple agents split responsibilities: Goal Agent defines strategy, Constraint Agent validates requirements.
2. Search for Jobs	Script opens predefined websites and searches exact keywords only.	Recommendation model predicts relevant jobs using learned patterns.	LLM can summarize search results but usually relies on external tools for browsing.	Agent autonomously browses LinkedIn, company sites, and job boards using tools/APIs.	Search Agent continuously monitors multiple platforms in parallel and shares results with other agents.
3. Read Job Description	Parses fixed fields using regex or predefined templates. Breaks if format changes.	ML extracts entities like skills, salary, location, seniority.	LLM reads full descriptions and understands context and requirements.	Agent analyzes requirements, compares them with resume, and identifies skill gaps.	Analysis Agent collaborates with Resume Agent and Skill-Match Agent for deeper evaluation.
4. Match Candidate Skills	Exact keyword matching only. "Python" matches only if written exactly.	ML predicts job-fit score from historical hiring data.	LLM reasons semantically: "FastAPI experience relates to backend API development."	Agent ranks jobs by fit, salary, growth potential, and interview probability.	Ranking Agent, Career Strategy Agent, and Market Agent collaboratively optimize opportunities.
5. Tailor Resume	Uses predefined resume templates with static replacements.	ML selects resume sections likely to perform well.	LLM rewrites resume bullets for each role using contextual understanding.	Agent customizes resume automatically for every application.	Resume Agent collaborates with ATS Optimization Agent and Skills Agent to maximize success rate.
6. Write Cover Letter	Uses rigid mail templates with placeholders.	ML predicts effective wording based on prior successful applications.	LLM generates personalized cover letters in natural language.	Agent writes, reviews, edits, and adapts tone based on company culture.	Writing Agent, Personalization Agent, and Quality Review Agent collaborate before submission.
7. Fill Application Form	Hardcoded automation fills predefined fields only.	ML helps classify field types.	LLM can understand field meanings but usually cannot act alone.	Agent autonomously navigates websites, uploads files, answers questions, and submits forms.	Execution Agent handles applications while Validation Agent checks for errors before submission.

Evolution of Intelligent Systems: From Traditional Programming to Agentic AI

Goal: Apply for a Software Engineer job

Step	Traditional Programming	Machine Learning	LLM / Chatbot	AI Agent	Agentic AI
8. Handle Unexpected Situations	Fails if page layout changes or CAPTCHA appears.	ML detects anomalies but cannot reason deeply.	LLM suggests possible actions conversationally.	Agent adapts dynamically: “Website changed layout; locate submit button another way.”	Recovery Agent collaborates with Execution Agent to resolve failures autonomously.
9. Communicate with Recruiters	Sends predefined automated emails only.	ML predicts best response timing.	LLM drafts professional recruiter replies.	Agent replies autonomously, schedules interviews, and manages follow-ups.	Communication Agent manages recruiter interaction while Calendar Agent handles scheduling.
10. Prepare for Interviews	Cannot reason about interview questions.	ML recommends likely interview topics.	LLM explains concepts, generates coding questions, and mocks interviews.	Agent creates personalized study plans and tracks preparation progress.	Coaching Agent, Coding Agent, and Behavioral Interview Agent collaborate continuously.
11. Track Progress	Logs applications in spreadsheets.	ML predicts probability of interview callbacks.	LLM summarizes status updates conversationally.	Agent continuously monitors application status and reminds user of deadlines.	Monitoring Agent coordinates all activities and updates strategy dynamically.
12. Decision Making	Fully rule-driven.	Statistical predictions only.	Language-based reasoning.	Goal-oriented autonomous reasoning and execution.	Distributed collaborative intelligence across multiple agents.
13. Human Involvement	Very high.	High.	Medium.	Low.	Minimal supervision needed.
14. Real-World Limitation	Extremely rigid and fragile.	Needs large training datasets.	May hallucinate or lack persistence.	Requires safeguards, permissions, and monitoring.	Complex orchestration and higher infrastructure cost.

LLM applications

Penetrative AI: Making LLMs Comprehend the Physical World

Huatao Xu[†], Liying Han[§], Qirui Yang^{*}, Mo Li^{*,†}, Mani Srivastava[§],

^{*}The Hong Kong University of Science and Technology,

[†]Nanyang Technological University, [§]University of California Los Angeles



Paper



arXiv



Code

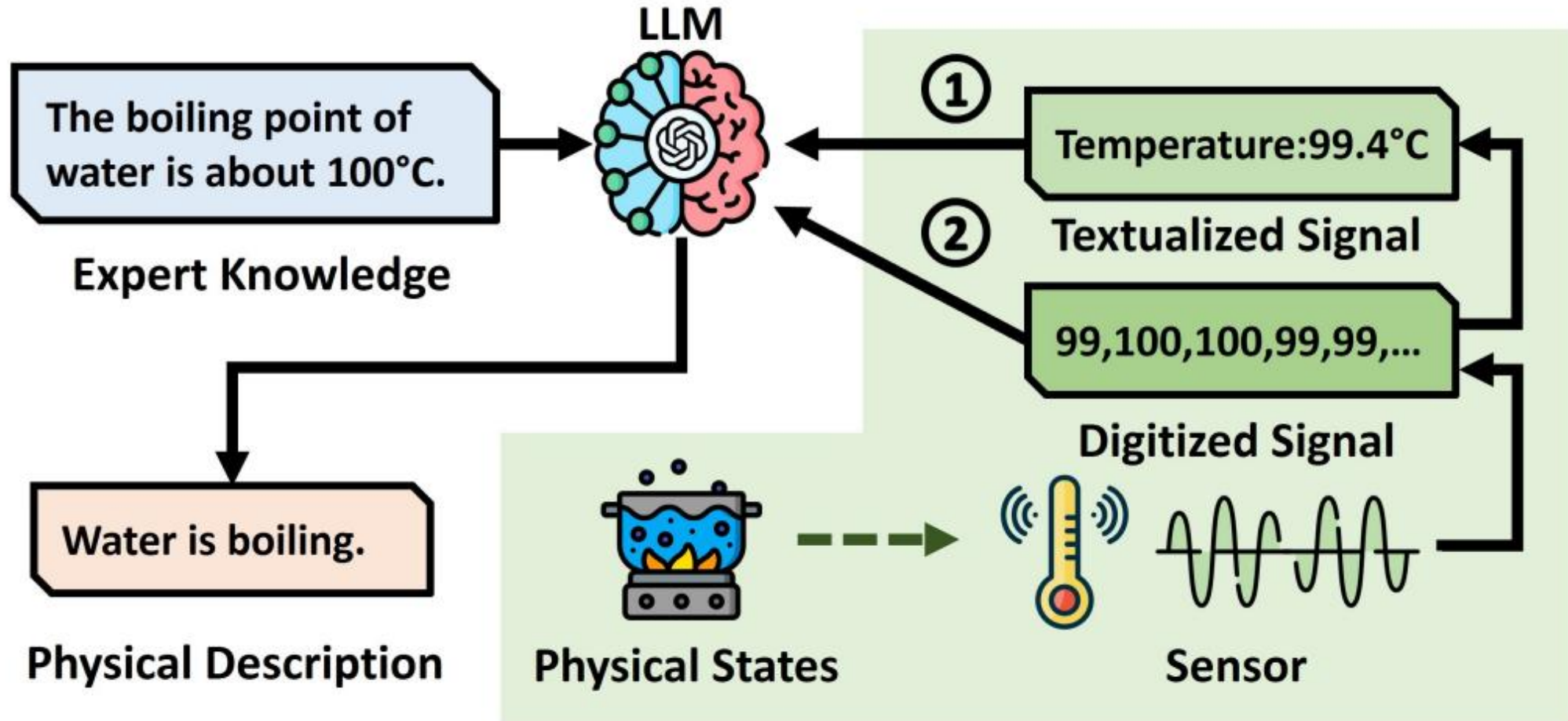
References

<https://github.com/dapowan/Penetrative-AI>

<https://arxiv.org/pdf/2310.09605>

Penetrative AI

- LLMs can be extended to interact with and reason about the physical world through IoT sensors and actuators.
- "Penetrative AI" where LLMs serves as a foundation model, seamlessly integrated with the Cyber-Physical Systems for perceiving & intervening in the physical world.



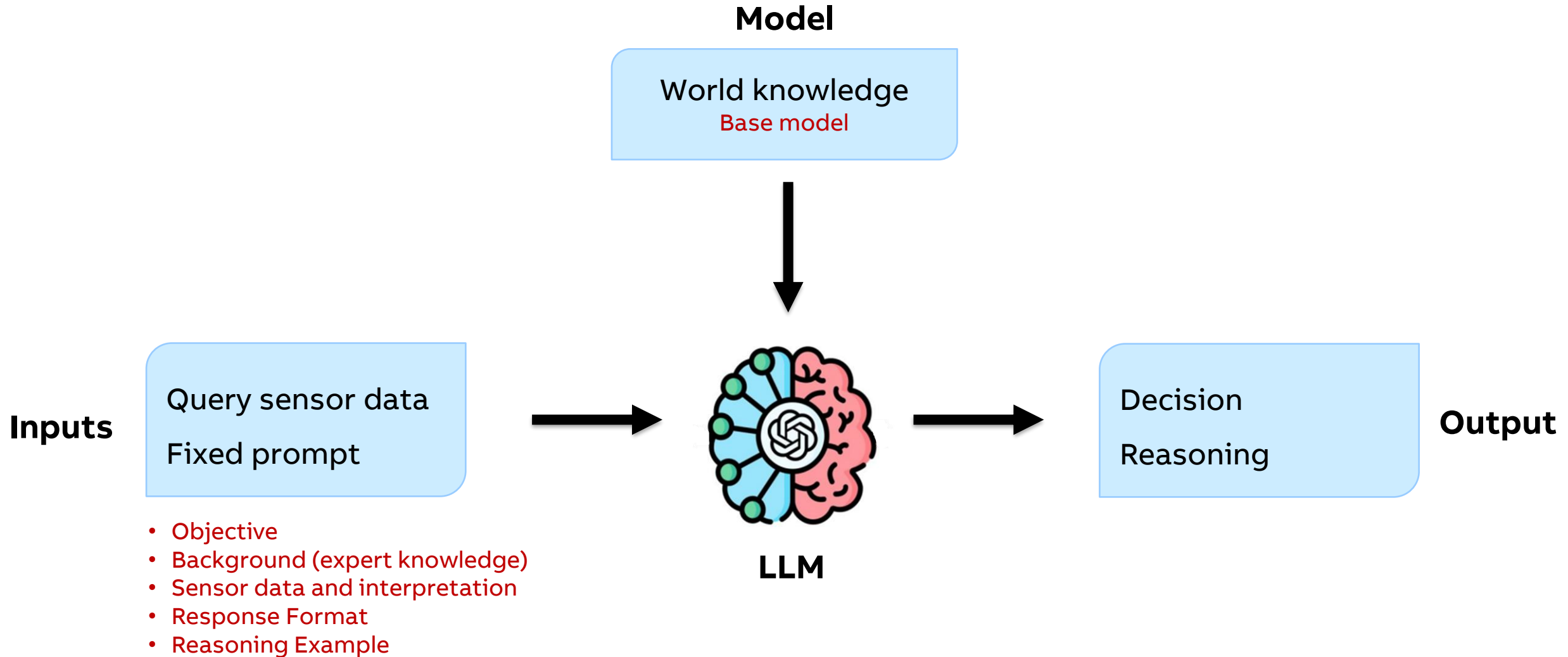
Elements of basic prompts

Component	Description	Example
1. Role / Identity	Defines <i>who</i> the AI should act as	<i>"You are a travel agent."</i>
2. Task / Goal	What you want the AI to do	<i>"Plan a 3-day beach vacation."</i>
3. Context / Input	Background info or specific data to guide the response	<i>"I have a \$1,000 budget. I like relaxing, not adventure."</i>
4. Constraints / Rules	Limits or boundaries for the output	<i>"Keep it under \$1,000. Avoid international travel."</i>
5. Output Format	Preferred structure, tone, or medium of the response	<i>"Present it as a bullet list with day-wise breakdown."</i>
6. Examples (optional)	Show what kind of answer you're expecting (especially useful for Gen AI)	<i>"Example: Day 1 – Arrival, Beach Walk, Sunset Dinner"</i>

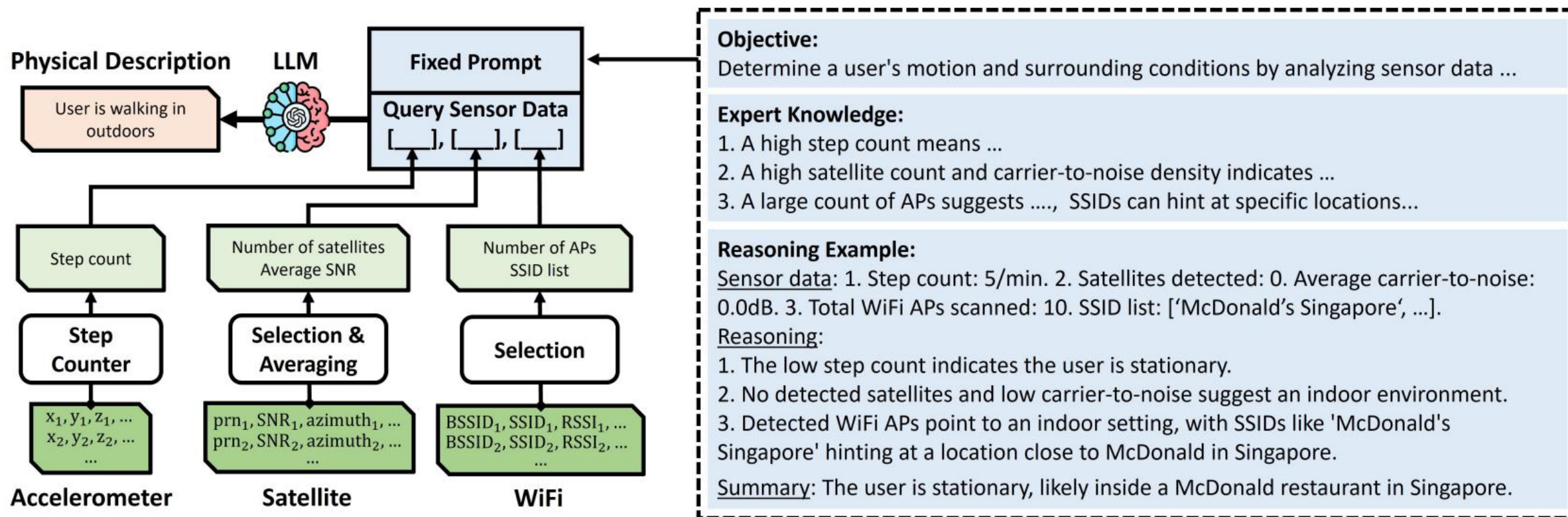
Full Prompt

"You are a travel agent. Plan a 3-day beach vacation for me. I have a \$1,000 budget, and I prefer calm activities like reading and swimming. No international travel. Please present the plan as a bullet-point list, one day at a time."

Building blocks



User activity sensing with LLMs Use case



Sampling frequency

- Accelerometer 100 Hz
- Satellite & Wi-Fi 0.2 Hz

Window size 10-45 seconds

- **PRN** Pseudo random noise = Unique identifier for satellites in systems like GPS
- **SNR** Signal to noise ratio = Ratio indicating the quality of the signal received from the satellite
- **Azimuth** = The horizontal angle indicating the direction of the satellite from the observer's location, measured clockwise from true north.
- **SSID** Service set identifier = The name of a Wi-Fi network that identifies it to users
- **RSSI** Receiver strength indicator = A measure of the strength of the received Wi-Fi signal, with higher (less negative) values indicating stronger signals

User activity sensing with LLMs Use case II

Input Sensor Data

1. Step count: 105.0/min.
2. Satellites detected: 0. Carrier-to-noise: 0.0dB.
3. Total WiFi APs scanned: 31. SSID list:
['.EastPointCity', 'WiFi.HK via EPCC ', '3DG Jewellery', '3HK Pro3 5G', '5G私家寬頻', 'citicall01', 'CSL WiFi-Roam', 'H158-381_6c49', 'CENTALINE FREE WIFI', 'MiHome', 'SASA_LIVE', '_BWSHOP_', ...]



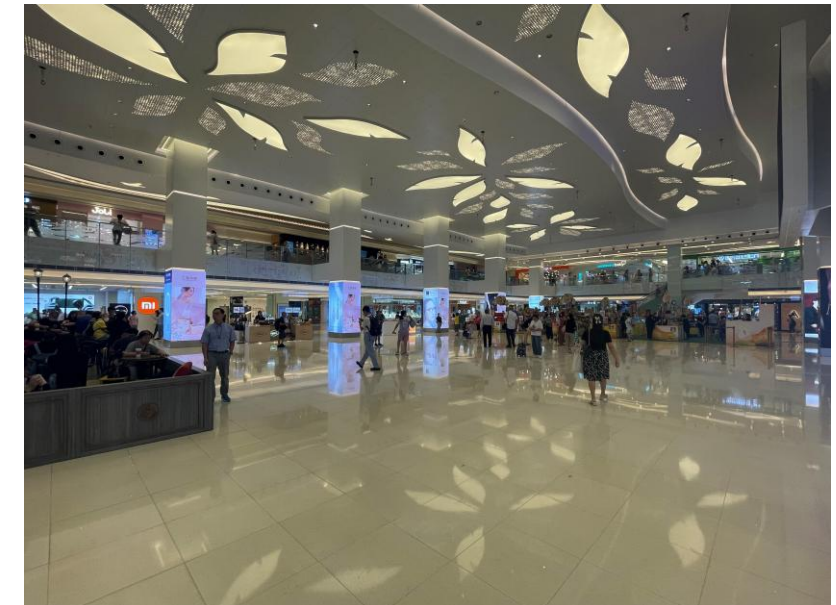
Response from ChatGPT-4

Reasoning:

1. Step count: 105.0/min. The high step count indicates that the user is likely walking, suggesting movement and active state.
2. Satellites detected: 0. Carrier-to-noise: 0dB. The absence of detected satellites and a carrier-to-noise ratio of 0 dB strongly suggest that the user is in an indoor environment, as satellite signals are usually weak or non-existent indoors.
3. Total WiFi APs scanned: 31. The detection of 31 WiFi Access Points supports the inference of an indoor environment. Modern buildings usually have a high density of WiFi networks. Analyzing the SSID list, such as '.EastPointCity', 'WiFi.HK via EPCC ', '3DG Jewellery', etc., it is reasonable to speculate that the user might be inside a shopping mall or a commercial area, possibly in Hong Kong, considering SSIDs like 'WiFi.HK via EPCC' and '3HK Pro3 5G'. 'EastPointCity' could be the name of a specific location or a mall.

Summary:

The user is likely walking inside a commercial building or a shopping mall, possibly located in Hong Kong or a region where there is a presence of Hong Kong-based telecom and WiFi services. The active and diverse WiFi networks and the absence of satellite signals confirm the indoor setting, and the high step count confirms the walking motion.



User activity sensing with LLMs Use case III



Input Sensor Data

1. Step count: 3.0/min.
2. Satellites detected: 0. Carrier-to-noise: 0.0dB.
3. Total WiFi APs scanned: 9. SSID list: ['NESL-5', 'eduroam', 'UCLA_WEB', 'UCLA_WIFI', 'AlphaExp5G', 'AlphaExp', 'GL-AR150-ak', 'nesl-AP', ...]

Summary: The user is stationary and is likely inside a building on the UCLA campus or in its vicinity.



Input Sensor Data

1. Step count: 0.0/min.
2. Satellites detected: 0. Carrier-to-noise: 0.0dB.
3. Total WiFi APs scanned: 5. SSID list: ['Mada's Samsung', '#WiFi@Changi', '#WiFi@Changi', '#WiFi@Changi', 'Eff'].

Summary: The user is stationary, and they are likely inside or close to Changi Airport in Singapore.

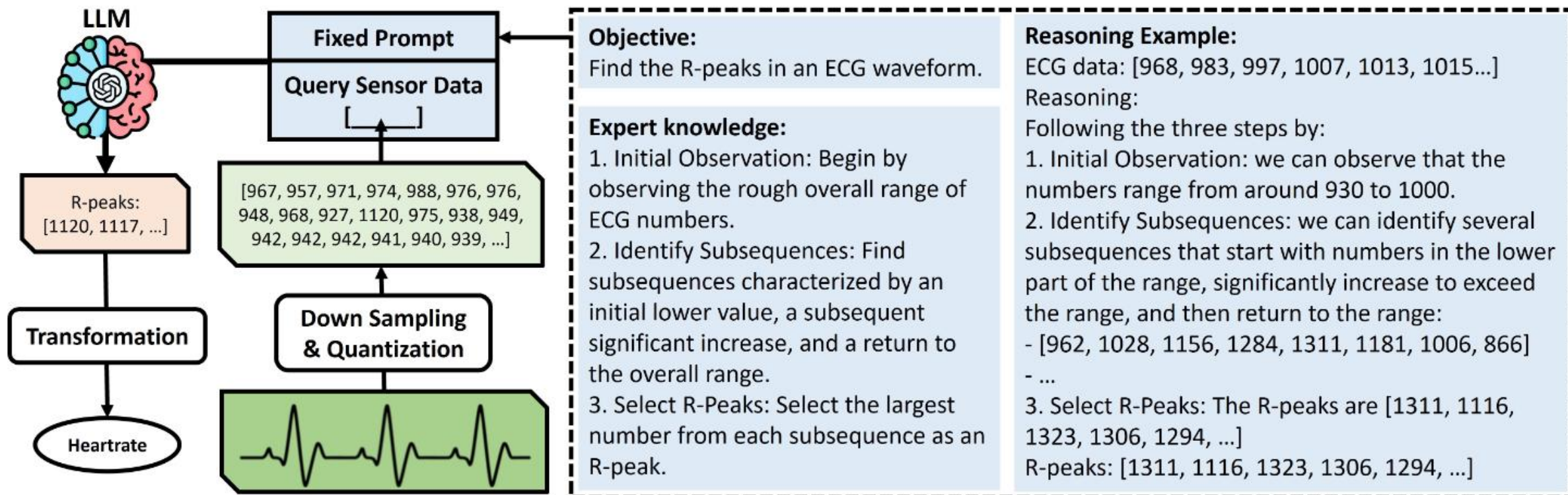


Input Sensor Data

1. Step count: 118.0/min.
2. Satellites detected: 6. Carrier-to-noise: 36.37dB.
3. Total WiFi APs scanned: 0. SSID list: [].

Summary: The user is likely walking in an outdoor environment.

User activity sensing with LLMs Use case I heartbeat detection



Sampling frequency

- Accelerometer 360 Hz
- Down sampled at 36 Hz

Window size 5 seconds

[Penetrative-AI/demo/heartbeat_detection.md at main · dapowan/Penetrative-AI · GitHub](#)

User activity sensing with LLMs Use case I heartbeat detection

Prompt

1. [Objective]
2. [Natural language algorithm]
3. [Examples]
4. ECG Data:
ECG data: [945 959 964 961 960 961 954 956 953 957
952 975 970 980 945 963 923 994 1059 915 960 931 953
935 953 938 945 930 953 960 976 966 973 963 972 961
972 961 974 970 995 982 995 953 986 921 1109 1028
931 977 947 970 955 967 957 965 953 972 977 986 976
983 971 975 966 972 964 973 968 991 981 992 955 976
937 985 1078 929 964 945 958 947 958 951 956 950 954
961 975 967 971 966 966 961 961 955 961 954 967 973
977 967 955 944 953 919 1076 959 929 943 935 938 934
939 934 934 923 932 948 955 950 953 948 948 944 947
941 947 943 953 960 965 953 946 936 939 905 1078 960
926 943 935 941 939 945 940 936 931 948 962 963 964
962 961 959 959 956 959 957 963 975 982 976 963 951
955 915 1129 986 935 954 944 953 949 955 950 952 950
963 973]

Response from ChatGPT-4

Reasoning:

Following the three steps by:

1. Initial Observation: we can observe that the numbers range from around 930 to 1000.

2. Identify Subsequences: we can identify several subsequences that start with numbers in the lower part of the range, significantly increase to exceed the range, and then return to the range:

- [923, 994, 1059, 915, 960]

- [921, 1109, 1028, 931, 977]

- [937, 985, 1078, 929, 964]

- [919, 1076, 959, 929, 943]

- [905, 1078, 960, 926, 943]

- [915, 1129, 986, 935, 954]

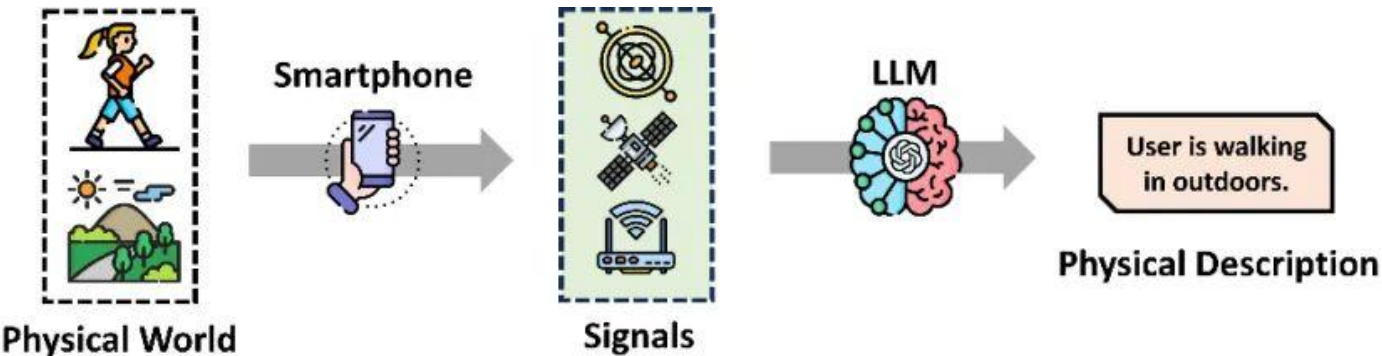
3. Select R-Peaks: The R-peaks are [1059, 1109, 1078, 1076, 1078, 1129]

R-peaks: [1059, 1109, 1078, 1076, 1078, 1129]

Ground truth

R-peaks: [1059, 1109, 1078, 1076, 1078, 1129]

Case study : Penetrative AI



Query Sensor Data

- 1. Step count: 105.0/min.
- 2. Satellites detected: 0. Carrier-to-noise: 0.0dB.
- 3. Total WiFi APs scanned: 22. SSID list: ['EastPointCity', 'WiFi.HK via EPCC', '3DG Jewellery', '3HK Pro3 5G', '5G私家寬頻', 'citical01', 'CSL WiFi-Roam', 'H158-381_6c49', 'CENTALINE FREE WIFI', 'MiHome', 'SASA_LIVE', '_BWSHOP_', ...]

ChatGPT

The user is walking, most likely indoors in a commercial or retail area, possibly located in Hong Kong.



Query Sensor Data

- 1. Step count: 3.0/min.
- 2. Satellites detected: 0. Carrier-to-noise: 0.0dB.
- 3. Total WiFi APs scanned: 9. SSID list: ['NESL-5', 'eduroam', 'UCLA_WEB', 'UCLA_WIFI', 'AlphaExp5G', 'AlphaExp', 'GL-AR150-ak', ...]

ChatGPT

The user is stationary and is likely inside a building on the UCLA campus or in its vicinity.



Query Sensor Data

- 1. Step count: 0.0/min.
- 2. Satellites detected: 0. Carrier-to-noise: 0.0dB.
- 3. Total WiFi APs scanned: 5. SSID list: ['Mada's Samsung', '#WiFi@Changi', '#WiFi@Changi', '#WiFi@Changi', 'Eff']

ChatGPT

The user is stationary, and they are likely inside or close to Changi Airport in Singapore.

Anomaly detection using LLMs

Objective:

Identify anomalies in the temperature variation of a heat exchanger's inner surface using synthetic data.

Background Knowledge:

- **Initial Observation:**

Start by observing the general range and trend of the temperature data. This helps establish what is considered "normal" behavior.

- **Identify Normal and Anomalous Behavior:**

- ❖ **Normal behavior:**

Stable temperature readings that fluctuate within a defined range.

- ❖ **Anomalous behavior:**

Significant deviations from the normal range, which can be either sudden spikes or drops in temperature.

- **Identify Anomalous Regions:**

Look for subsequences where the temperature deviates significantly from the normal range and then returns to it. Anomalies could be identified by:

- ❖ Sudden and large temperature spikes.

- ❖ Sudden and large temperature drops.

- **Response Format:**

- ❖ **Reasoning:** Provide a detailed explanation of how anomalies were identified based on the steps mentioned above.

- ❖ **Anomalies:** List the identified anomalous temperatures.

Anomaly detection using LLMs

Synthetic Data Example

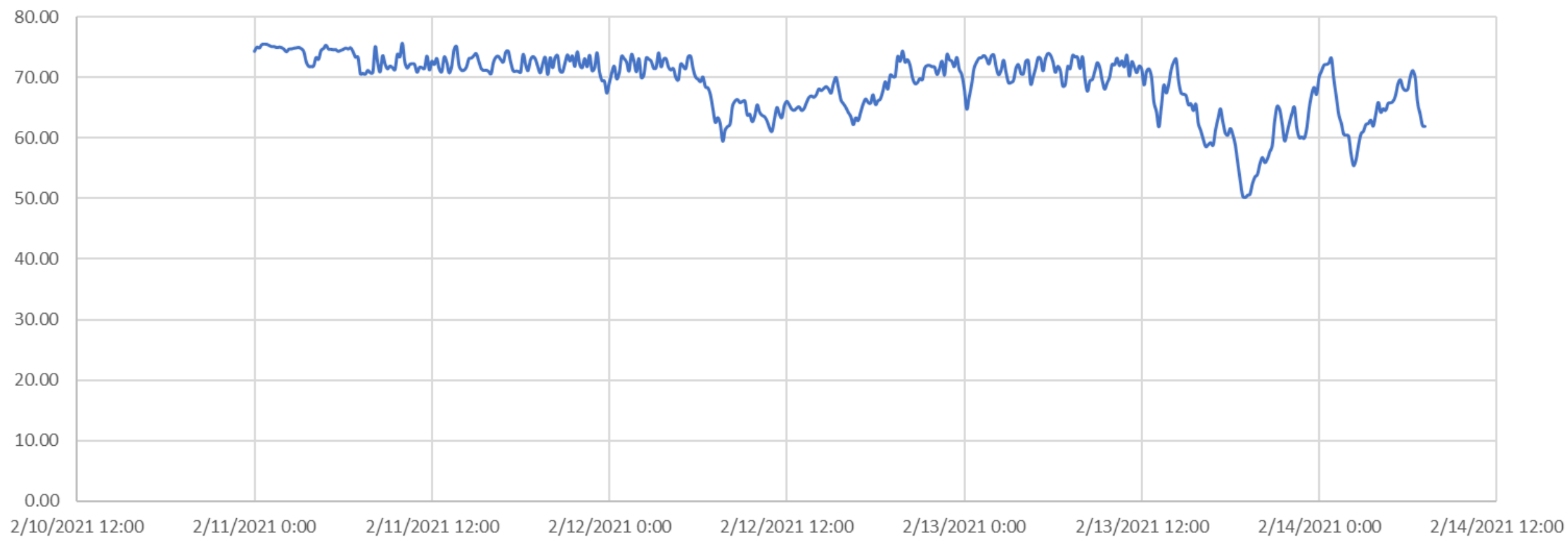
Temperature Data:

[75, 76, 74, 77, 75, 78, 79, 150, 77, 76, 75, 73, 72, 150, 75, 76, 77, 78, 150, 75, 74, 76, 75, 73, 74, 77, 150, 78, 76, 75, 73, 72, 75, 76]

Reasoning:

- **Initial Observation:**
The temperature readings range between 72 and 79 degrees Celsius, which is considered normal. Any value significantly above or below this range could be considered an anomaly.
- **Identify Normal and Anomalous Behavior:**
 - ❖ **Normal behavior:** Temperatures in the range of 72 to 79.
 - ❖ **Anomalous behavior:** Temperatures significantly deviating from this range, such as 150 degrees.
- **Identify Anomalous Regions:** Observing the data, we can identify several instances where the temperature spikes to 150 degrees, which is significantly higher than the normal range.
- **Response Format:**
 - ❖ **Reasoning:**
By observing the data, we can see that temperatures mostly stay within the 72 to 79 range. However, there are sudden spikes to 150 degrees, which are clear anomalies.
 - ❖ **Anomalies:**
150, 150, 150, 150

Anomaly detection using LLMs



LLMs beyond Chatgpt for engineering: Leo CAD Copilot, Urban GPT etc.



Leo™

<https://www.getleo.ai/>



<https://cesium.com/platform/cesiumjs/>

UrbanGPT

<https://urban-gpt.github.io/>



<https://www.nvidia.com/en-in/industries/aec/>

zenerate

<https://www.zenerate.ai/>

<https://www.youtube.com/watch?v=0UvmQDn7t-o&t=10s>



Giraffe

<https://www.giraffe.build/>

<https://www.youtube.com/shorts/srQ-LIA37ZE>

Videos by OpenAI on ChatGPT's visual/audio/video capabilities

[Live demo of GPT-4o coding assistant and desktop app \(youtube.com\)](#)

2.22 to 2.36

ChatGPT interprets time series temperature plot

[Live demo of GPT-4o vision capabilities \(youtube.com\)](#)

0.58 to 2.48

ChatGPT analyzes the uploaded image to identify any linear equations present.

It then utilizes its mathematical understanding to solve these equations step by step.

Users receive solutions or guidance based on the detected equations, enhancing their problem-solving experience.

Chatbots

- **Simulate human-like conversations** using NLP techniques to answer questions and assist users.
- **LLM-powered chatbots can understand context, follow instructions, and generate detailed responses**, unlike traditional rule-based bots.
- **Modern chatbot platforms such as ChatGPT, Google Gemini, and Microsoft Copilot etc.**

Activity 1 — Research Idea Brainstorming

Goal: Show how a chatbot can help generate research ideas.

Prompt:

You are an expert research assistant in computer science. Suggest 5 research topics in the field of Artificial Intelligence for smart cities. For each topic include:

- problem statement
- possible methodology
- potential impact.

Activity 2 — Methodology Design

Goal: Show how LLM chatbots help design research methodology.

Prompt:

You are a research advisor. Suggest a research methodology for developing a machine learning model to detect road accidents using CCTV video data. Include:

- dataset collection
- preprocessing
- model selection
- evaluation metrics.

Code generation & build interactive data apps

- **LLMs can generate code** in languages such as Python, Java, C++, and MATLAB based on instructions.
- Assist engineers in **rapid prototyping, debugging, and optimizing algorithms**, significantly reducing development time.
- Build **interactive data applications for visualization, analysis, and dashboards** using frameworks like Streamlit or Flask.

Activity 1 — Algorithm Implementation

Demonstrate how LLMs implement algorithms.

Prompt

Write a Python program to implement the K-Means clustering algorithm from scratch using NumPy. Also visualize the clusters using matplotlib. Explain the steps briefly.

Activity 2 — Simulation Code

Prompt

Write a Python program that simulates projectile motion. The program should:

- take initial velocity and angle as input
- compute trajectory
- plot the motion graph.

Code generation & build interactive data apps

Activity 3 — Build an Interactive Data App

Tell ChatGPT to generate a **Streamlit data dashboard**.

Prompt

You are a data scientist and software developer. Generate a complete Python Streamlit application for Exploratory Data Analysis (EDA). The application should allow users to upload a CSV dataset and perform interactive data analysis. Requirements:

1. File Upload

- Allow users to upload a CSV dataset using Streamlit file uploader.
- Display dataset shape (rows and columns).
- Display the first 10 rows of the dataset.

2. Dataset Information

- Show column names and data types.
- Show number of missing values per column.

3. Summary Statistics

- Display descriptive statistics (mean, median, std, min, max).
- Show separate summaries for numerical & categorical columns.

4. Data Visualization

Allow the user to select columns and generate visualizations:

Numerical columns: Histogram, Box plot

Categorical columns: Bar chart showing frequency counts

Relationships between variables:

- Scatter plot between two selected numerical columns
- Correlation heatmap for numerical features

5. Missing Data Visualization

- Plot missing values using a heatmap.

6. Interactive Controls

Use Streamlit widgets such as:

- selectbox
- multiselect
- slider

7. Libraries to use

Use the following Python libraries:

- pandas
- matplotlib
- seaborn
- streamlit

8. Code Quality

- Write clean and well-commented code.
- Organize with clear sections with Streamlit headers.
- Ensure the app runs without errors.

9. Instructions

Provide instructions on how to run the app locally:

- pip install commands
- command to run Streamlit app

Output:

Provide the complete Python code in one file named app.py.

Content Generation

- **Generate structured academic content** such as research proposals, literature summaries, and technical reports.
- Create **assignments, tutorials, quizzes, and question papers** aligned with course outcomes.
- LLMs can assist in **developing learning activities, case studies, and project ideas** for engineering and technical courses.
- AI tools like ChatGPT and Google Gemini help researchers and educators produce high-quality content quickly.

Activity 1 — Generate a learn-by-doing interactive tutorial Prompt

You are an AI tutor teaching ML to engineering students. Create an interactive "learn-by-doing" tutorial to explain Decision Tree classification with example of classifying traffic congestion. Make the tutorial engaging & interactive.

The tutorial should follow these rules:

1. Start from zero knowledge.
2. Explain concepts step by step.
3. After every concept, give a small activity for learner.
4. Ask the learner to think, calculate, or make a decision before revealing the answer.
5. Keep explanations simple and intuitive.

Structure the tutorial as follows:

Step 1: Problem understanding

Explain classifying traffic congestion problem
(Low/Medium/High)

Step 2: Features and dataset

Introduce example features: Time of day, Rain (Yes/No),
Accident (Yes/No), Traffic volume

Give a small dataset table, ask learner to observe patterns.

Step 3: What is a Decision Tree

Explain the concept of splitting data based on conditions.

Activity: Ask the learner which feature should be the first split to classify congestion.

Step 4: Understanding entropy & information gain (intuitive way)

Explain impurity using simple examples.

Activity: Ask the learner to compare two possible splits and decide which gives better separation.

Step 5: Building the decision tree step by step

Construct the tree gradually.

Activity: Ask the learner to choose next split based on data.

Step 6: Prediction

Give a new traffic scenario and ask the learner to follow the tree to predict congestion level.

Step 7: Implementation

Provide a Python example using scikit-learn to train a decision tree model on a small traffic dataset.

Step 8: Reflection

Ask learner questions to test understanding of features, decision nodes, leaf nodes, classification.

Multimodal (Text, documents, Images, Audio, Video) processing –

Summarize, extract, classify, compare

Activity 1 — Document Summarization

Upload a **research paper or PDF**.

Prompt:

Summarize this document in 5 bullet points.
Also list the main objectives and key findings.

Activity 2 — Information Extraction from Documents

Upload a **research article or report**.

Prompt:

Extract the following information from this document:

1. Problem statement
2. Methodology used
3. Dataset mentioned
4. Key results

Activity 3 — Image Understanding

Upload an **image of traffic congestion or a street scene**.

Prompt:

Analyze this image and describe what is happening.
Identify possible factors contributing to traffic congestion.

Activity 4 — Compare Two Documents

Upload two short documents or research abstracts.

Prompt:

Compare these two documents and highlight:

1. Similarities in methodology
2. Differences in objectives
3. Key contributions of each work.

Activity 5 — Audio Processing

Upload an **audio lecture or speech recording**.

Prompt:

Transcribe this audio and summarize the key points discussed in the lecture.

Activity 6 — Video Understanding

Upload a short **traffic surveillance video**.

Prompt:

Analyze this video and identify events that may cause traffic congestion.
Provide a brief explanation.

Dataset generation using LLMs for research and development

Activity 1 — Synthetic Dataset for ML Training

Use case: create data when real data is unavailable.

Prompt:

Generate a synthetic dataset for traffic congestion prediction. Columns: Time_of_day, Rain, Accident, Traffic_volume, Congestion_level. Create 50 rows in CSV format with realistic values.

Activity 2 — Balanced Dataset for Classification

Use case: remove class imbalance.

Prompt:

Create a dataset for binary classification of traffic congestion. Columns: vehicle_count, average_speed, rain, accident, congestion_label (0 or 1). Generate 100 samples with balanced classes (50 congested and 50 non-congested).

Activity 3 — Rare Event Dataset

Use case: simulate uncommon situations.

Prompt:

Generate 30 traffic scenarios representing rare events such as: heavy rain, multiple accidents, road construction, sudden traffic spikes. Include congestion level labels.

Activity 4 — Edge Case Dataset (Corner Cases)

Use case: test algorithm robustness.

Prompt:

Generate 20 extreme traffic congestion scenarios where: traffic volume exceeds normal capacity, average speed drops below 5 km/h, multiple accidents occur. Include realistic sensor readings.

Activity 5 — Noisy Dataset

Use case: simulate real-world sensor errors.

Prompt:

Create a dataset for traffic sensors where some readings contain noise or missing values. Columns: vehicle_count, average_speed, occupancy, congestion_label. Include occasional missing values or slightly incorrect readings.

Activity 6 — Feature Expansion Dataset

Use case: explore new features for research.

Prompt:

Generate a traffic congestion dataset with additional features such as: weather_condition, road_type, nearby_event, traffic_signal_density. Create 50 samples.

Dataset generation using LLMs for research and development

Activity 7 — Realistic Dataset Based on Engineering Knowledge

You are an energy systems researcher.

Generate a realistic dataset for predicting solar power generation in a photovoltaic system.

Columns: timestamp,
solar_irradiance (W/m²),
panel_temperature (°C),
ambient_temperature (°C),
panel_efficiency (%),
power_output (kW)

Constraints:

- power_output should increase with solar_irradiance
- panel efficiency slightly decreases as panel temperature increases
- realistic irradiance range: 0-1000 W/m²
- generate 50 rows in CSV format

Activity 8 — Dataset Using First-Principles Formula

Generate a dataset for solar panel power generation using the formula:

Power_output =
Solar_Irradiance × Panel_Area
× Efficiency

Columns:

- solar_irradiance (W/m²)
- panel_area (m²)
- efficiency (%)
- power_output (kW)

Constraints:

- calculate power_output using the formula
- use realistic ranges for solar irradiance (200-1000 W/m²)
- generate 40 rows in CSV format.

Activity 9 — Dataset Inspired by Benchmark Dataset

Generate a synthetic dataset similar to solar power forecasting datasets used in renewable energy research.

Columns:

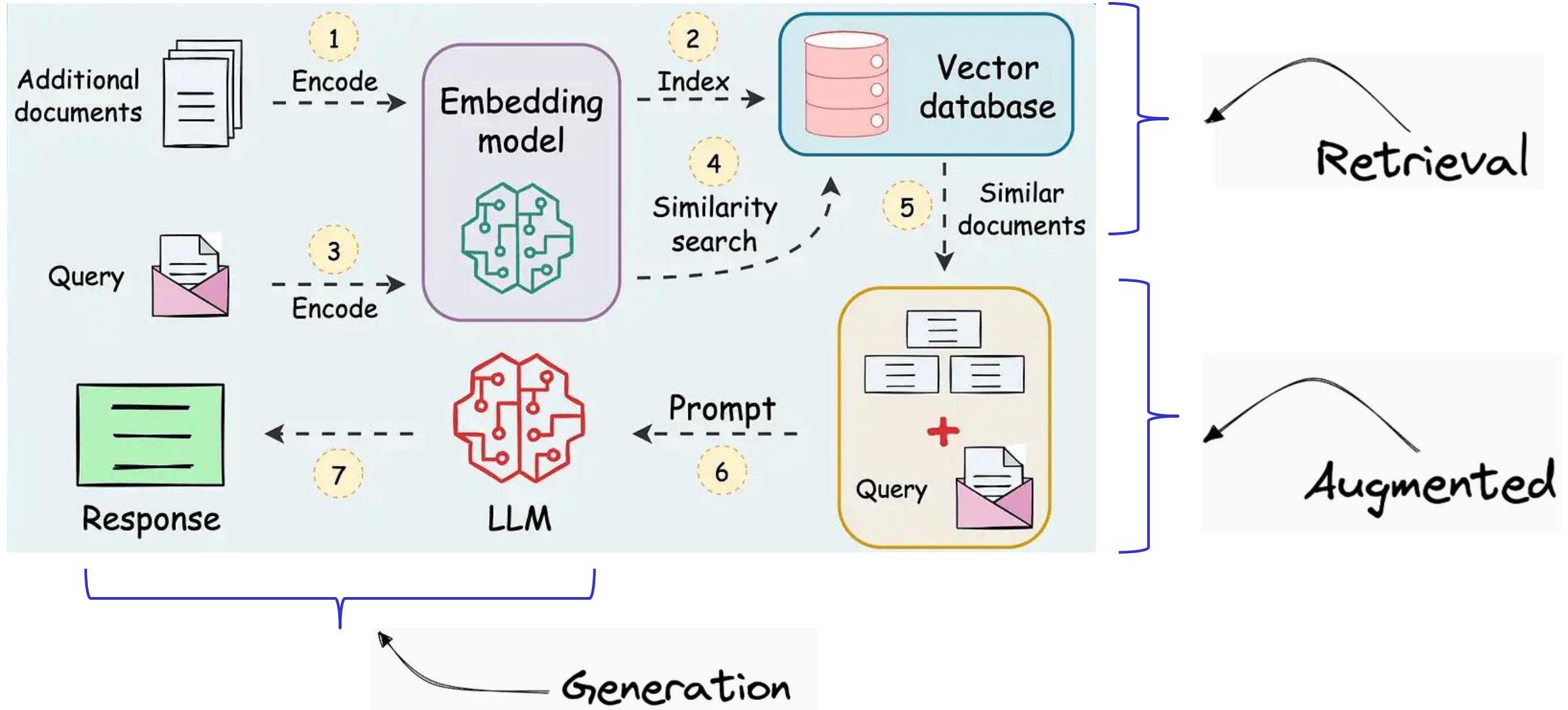
- timestamp
- solar_irradiance
- temperature
- humidity
- wind_speed
- solar_power_output

Generate 60 realistic rows that follow typical solar generation patterns during a day.

Knowledge Assistants (RAG)

Knowledge Assistants (RAG)

- **Retrieval Augmented Generation (RAG)** combines Large Language Models with external knowledge sources such as documents, databases, or websites.
- Instead of relying on pretrained knowledge, RAG **retrieves relevant documents and uses them to generate answers.**



Knowledge Assistants (RAG)

Activity 1 — Knowledge Assistant for Research Papers

Upload a **PDF research paper** in ChatGPT.

Prompt:

You are a research assistant.

Answer questions using only the information in the uploaded paper.

Question:

What problem does this paper address and what methodology is used?

Activity 2 — University Knowledge Assistant

Upload a **course syllabus or handbook**.

Prompt:

You are a university knowledge assistant.

Use the uploaded document to answer the following question:

What are the prerequisites for this course and how is the evaluation done?

Explain:

Universities can build AI assistants for academic information.

Activity 3 — Compare Multiple Documents

Upload **two research papers**.

Prompt:

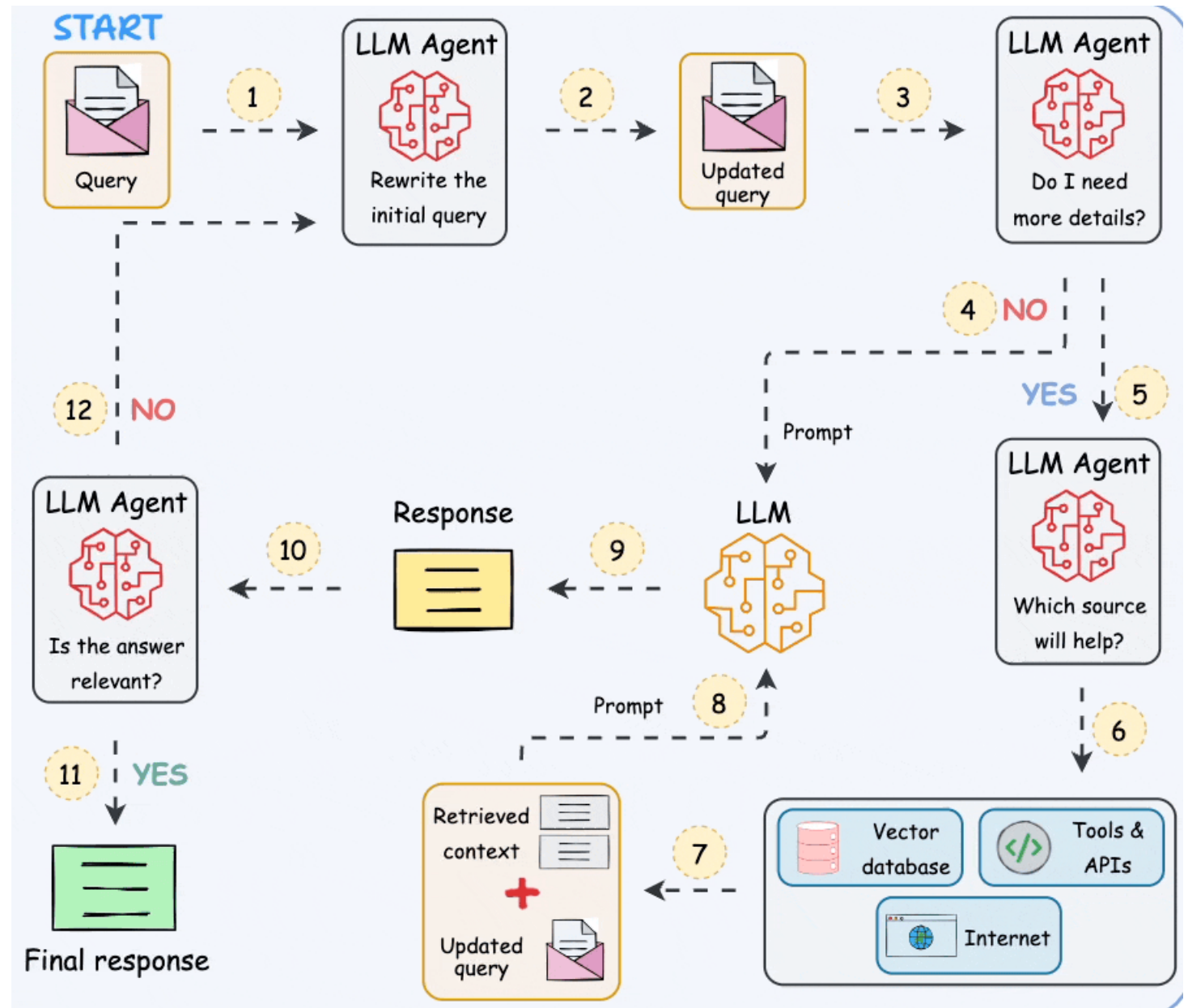
Compare the approaches proposed in the uploaded papers.

Explain:

1. differences in methodology
2. advantages of each approach

Agentic RAG

1. **User submits a query** to the system.
2. **LLM Agent rewrites or refines the query** to better capture the user's intent and improve retrieval.
3. The system checks **whether the query needs additional details or clarification**.
4. If **no additional information is required**, the query proceeds directly to the LLM for response generation.
5. If **additional knowledge is required**, the agent determines **which information source will be useful**.
6. The agent selects appropriate sources such as **vector databases, external APIs, or internet search**.
7. The system **retrieves relevant documents or contextual information** from these sources.
8. The **retrieved context is combined with the updated query** to form the final prompt.
9. The **LLM generates a response** based on the provided context and prompt.
10. The generated **response is evaluated by the agent** for quality and relevance.
11. If the answer is **relevant and satisfactory**, it is returned to the user.
12. If the answer is **not relevant**, the agent may refine the query and repeat the retrieval process.



Prompt engineering

Prompt engineering

A **prompt** is the message or instruction you give to an AI tool like ChatGPT to get a response.

Think of it as:

Asking a smart assistant to help you — the better your question, the better the answer.

Examples:

“Summarize this paragraph in simple words.”

“Write a 5-day workout plan for a beginner.”

The AI understands natural language — no coding needed!

Prompt Engineering is the skill of crafting **clear, precise, and powerful prompts** to get exactly what you want from AI.

It's like learning to **ask smart questions** so the AI gives **smart answers**.

Why it matters:

- Better prompts → Better results
- Avoids confusion or wrong output
- Saves time by getting it right the first time

Prompt engineering

Tips for Writing Better Prompts

Be specific → *“Write a study plan for 12th-grade physics students”* is better than *“Make a study plan”*

Set a tone or role → *“Act like a friendly teacher and explain this concept to a 10-year-old.”*

Give context → *“Based on these marks: 45 in Math, 70 in Science...”*

Add format if needed → *“List steps in bullet points.”*, *“Write in table format.”*

Basic Prompt

“Make a quiz”

“Write feedback”

“Explain photosynthesis”

“Write a letter”

Engineered Prompt

“Make a 5-question quiz on Grade 7 Algebra with answers.”

“Write polite feedback for a student who scored low in Algebra.”

“Explain photosynthesis in 3 sentences for a 6th-grade student.”

“Write a formal leave letter to the principal for 2 days’ leave.”

Prompt engineering

Prompt Engineering is the skill of crafting **clear, precise, and powerful prompts** to get exactly what you want from AI.

It's like learning to **ask smart questions** so the AI gives **smart answers**.

Why it matters:

- Better prompts → Better results
- Avoids confusion or wrong output
- Saves time by getting it right the first time

Elements of basic prompts

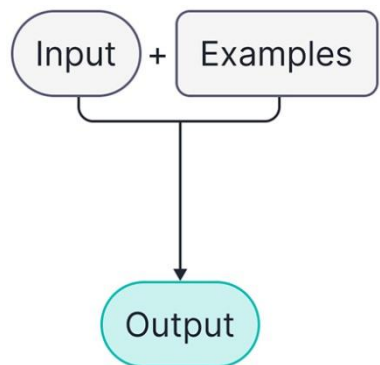
Component	Description	Example
1. Role / Identity	Defines <i>who</i> the AI should act as	<i>"You are a travel agent."</i>
2. Task / Goal	What you want the AI to do	<i>"Plan a 3-day beach vacation."</i>
3. Context / Input	Background info or specific data to guide the response	<i>"I have a \$1,000 budget. I like relaxing, not adventure."</i>
4. Constraints / Rules	Limits or boundaries for the output	<i>"Keep it under \$1,000. Avoid international travel."</i>
5. Output Format	Preferred structure, tone, or medium of the response	<i>"Present it as a bullet list with day-wise breakdown."</i>
6. Examples (optional)	Show what kind of answer you're expecting (especially useful for Gen AI)	<i>"Example: Day 1 – Arrival, Beach Walk, Sunset Dinner"</i>

Full Prompt

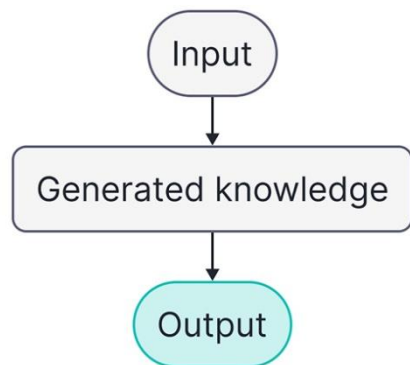
"You are a travel agent. Plan a 3-day beach vacation for me. I have a \$1,000 budget, and I prefer calm activities like reading and swimming. No international travel.

Please present the plan as a bullet-point list, one day at a time."

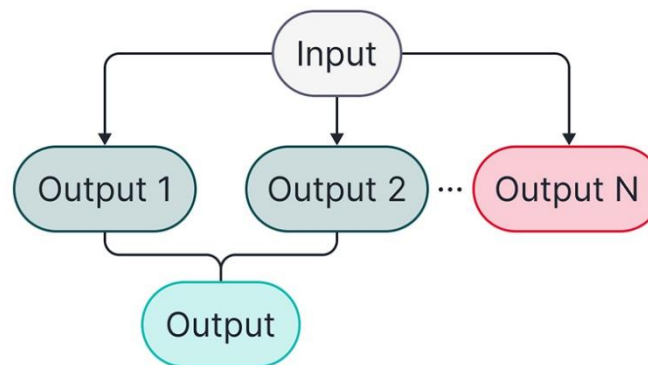
Prompt types



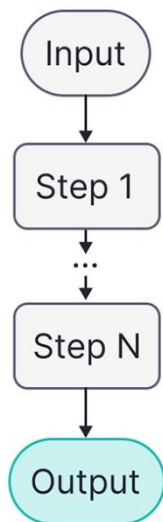
(a) Few-shot



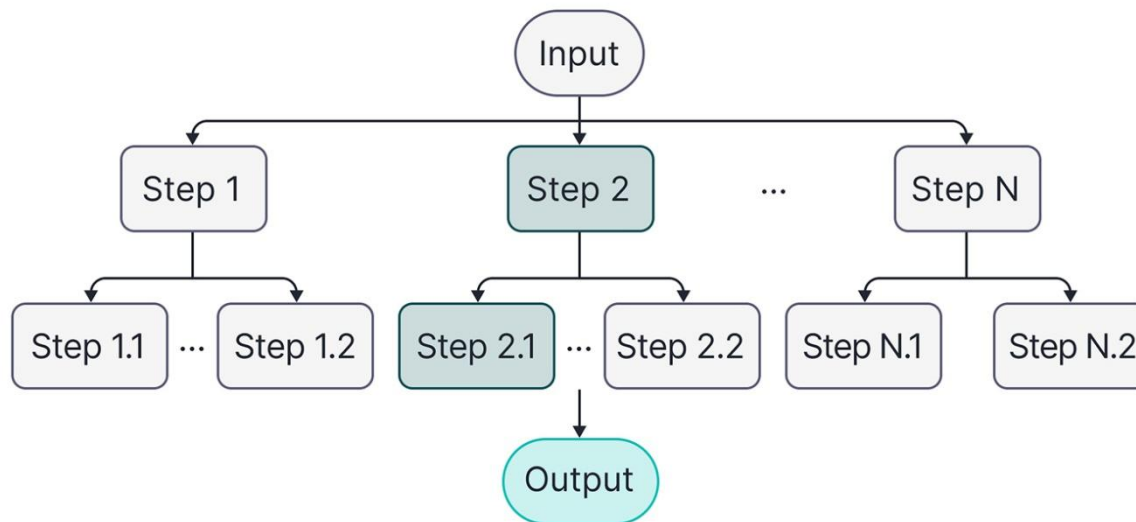
(b) Generated knowledge



(c) Self-consistency



(d) Chain-of-thoughts



(e) Tree-of-thoughts

Few-Shot Prompting Example - Robotic Arm Task

Problem:

“A car of mass 2 kg accelerates at 3 m/s². Find the force.”

Formula:

$$F = ma$$

Teacher does:

Shows examples:

$$m=1, a=2 \rightarrow F=2$$

$$m=2, a=2 \rightarrow F=4$$

Gives new problem

Student thinks:

“Same pattern $\rightarrow$ multiply $m \times a$ ”

Answer:

$$F = 2 \times 3 = 6 \text{ N}$$

Learns by copying examples

Generated Knowledge Prompting

Teacher asks:

1. “What is the formula?”
2. “What does mass mean?”
3. “What is acceleration?”

Student thinks:

- Force depends on mass and acceleration
- Uses rule $F = m \times a$

Answer:

$$F = 6 \text{ N}$$

Understand first, then solve

Self-Consistency Prompting

Teacher says:

“Solve it in different ways”

Student tries:

- Method 1: Direct formula → 6 N
- Method 2: Unit check → still 6 N
- Method 3: Recalculate → 6 N

Answer:

$F = 6 \text{ N}$ (confirmed)

Same answer repeated → more confidence

Chain-of-Thought Prompting

Teacher says:

“Show all steps”

Student writes:

1. Given: $m = 2 \text{ kg}$, $a = 3 \text{ m/s}^2$

2. Formula: $F = m \times a$

3. Substitute: $F = 2 \times 3$

4. Solve: $F = 6 \text{ N}$

Answer:

$F = 6 \text{ N}$

Clear step-by-step thinking

Tree-of-Thought Prompting

Teacher says:

“Think of different ways to solve”

Student explores:

Option 1: Direct formula $\rightarrow 6\text{ N}$

Option 2: Think physically (more mass + more acceleration $\rightarrow$ bigger force)

Option 3: Compare with other cases

Answer:

Best answer: 6 N

(Other ways support same result)

Explores different paths before deciding

Tree-of-Thought Prompting

Task: Choose the best college course plan by exploring multiple options and reasoning.

Prompt:

“I’m starting my career. Should I choose an Associate job, a Quality Testing role, or a Data Analyst position? Explore each briefly and recommend the best one.”

Tree of Thought Output:

Option	Pros	Cons	Growth Potential	Recommendation
Associate Job	Easy to enter, basic skills needed	Often non-technical, slow growth, limited technical exposure	☆☆	Only if domain-specific (e.g., finance)
Quality Testing (QA)	Good entry to tech, high demand, can shift to automation or dev roles	Repetitive if manual, less creativity than dev/analytics roles	☆☆☆☆	Good technical start
Data Analyst	In-demand, works with data tools (Excel, SQL, Python), strong salaries	Requires upskilling, steep initial learning curve	☆☆☆☆☆	Best choice for long-term growth

Final Recommendation:

*If you're technically inclined and want long-term growth, **Data Analyst** offers the best scope. QA is a safe tech start. Associate is less preferred unless it's with a high-growth path or in a domain of interest.*

Comparison between types

Prompt Type	How it Thinks	Real-Life Where Used
Few-shot	“I’ve seen this before”	◆ Coding assistants (GitHub Copilot) ◆ Chatbots trained with examples ◆ Lab manuals & tutorials
Generated Knowledge	“Let me figure out rules first”	◆ Research & R&D ◆ Design engineering ◆ AI systems learning new domains
Self-Consistency	“Let me try multiple times”	◆ Safety systems (aviation, medical AI) ◆ Simulations ◆ Financial predictions
Chain-of-Thought	“Let me solve step by step”	◆ Exams & education ◆ Debugging code ◆ Engineering calculations & reports
Tree-of-Thought	“Let me explore all options”	◆ Product design ◆ Architecture planning ◆ Strategy & decision-making systems

Structured Outputs

What are Structured Outputs?

- Outputs generated in a **well-defined format**
- Easy for **machines and software to process**
- Common in **AI systems, APIs, and automation tools**
- Helps ensure **consistency, reliability, and accuracy**

Examples:

- ☐ JSON
- ☐ XML
- ☐ CSV
- ☐ Schema-based responses

Importance in Modern AI Applications

- Enables **machine-readable responses**
- Improves **data processing automation**
- Reduces **errors in interpretation**
- Supports **integration with databases and applications**
- Essential for **AI pipelines and APIs**

Example use cases:

- ☐ Chatbots
- ☐ Data extraction systems
- ☐ Automated reporting

What are Structured Outputs?

Activity 1 — Converting Paragraphs into Tables

Real Use Case: Research Literature Review

In academic research, scholars read many research papers and summarize them into comparison tables for literature review sections in journals and theses.

Researchers often start with paragraph descriptions of papers and convert them into tables to compare studies.

Convert the following literature review paragraph into a table with columns:

Author	Year	Topic	Key Contribution
Sharma	2021	studied the use of machine learning for predicting student performance and found that decision tree models improved prediction accuracy.	
Rao	2022	analyzed deep learning methods for medical image classification and achieved high accuracy in tumor detection.	
Patel	2020	explored natural language processing techniques for sentiment analysis in social media data.	

What are Structured Outputs?

Activity 2 — Generating Structured Lists

Real Use Case: E-commerce Product Listings

E-commerce platforms like Amazon and Flipkart display product information as structured lists of features rather than long paragraphs.

Example: When sellers upload product descriptions, systems convert them into structured feature lists..

Convert the following product description into a structured list of product features.

Description:

The laptop comes with a 15.6 inch display, Intel i7 processor, 16GB RAM, 512GB SSD storage and a 10-hour battery life. It also supports WiFi 6 and has a backlit keyboard.

What are Structured Outputs?

Activity 3 — Standardized Report Format

Real Use Case: Medical Diagnostic Reports

Hospitals generate standardized diagnostic reports so every patient report follows the same structure.

Electronic health record systems used in hospitals (such as Epic Systems) organize reports using fixed sections like:

Patient Information, Symptoms, Diagnosis, Treatment Plan.

This standardization ensures doctors can quickly read and interpret reports.

Write a medical report using the following standardized format:

Patient Information

Symptoms

Diagnosis

Recommended Treatment

Case:

A 40-year-old patient reports fever, sore throat, and fatigue for three days. The doctor diagnosed viral flu and recommended rest and medication.

Output Formatting

What is Output Formatting?

Output formatting refers to organizing responses in a **specific structure**.

Examples:

Unstructured Output

Name: Ravi Kumar, Age: 35, Department: Computer Science

Structured Output (JSON)

```
{  
  "name": "Ravi Kumar",  
  "age": 35,  
  "department": "Computer Science"  
}
```

Benefits:

- ☐ Easy parsing
- ☐ Better automation
- ☐ Clear structure

JSON and Schema Responses

JSON (JavaScript Object Notation)

- ❑ Lightweight data format
- ❑ Human readable
- ❑ Widely used in APIs and AI models

Example:

```
{  
  "student": "Anita",  
  "course": "AI",  
  "marks": 92  
}
```

Schema-Based Responses

A schema defines the **expected structure of data**.

Example fields:

- ❑ name
- ❑ age
- ❑ department

JSON and Schema Responses

Activity 1 — API Response for Weather Data

Real Use Case

Weather apps get data from APIs that return information in **JSON format**.

Applications like Google Weather or AccuWeather receive structured JSON responses from weather servers.

Generate a JSON response for a weather API with the following fields:

city

temperature

humidity

weather_condition

City: Hyderabad

Temperature: 32°C

Humidity: 60%

Weather: Sunny

JSON and Schema Responses

Activity 2 — Chatbot Structured Response

Real Use Case

AI chatbots convert user messages into structured JSON so backend systems understand requests.

Customer service platforms like Zendesk use this approach.

Convert the following user request into a structured JSON response.

User request:

"I want to book a train ticket from Hyderabad to Bangalore tomorrow."

Fields:

intent

source_city

destination_city

date

Data Extraction

Process of **pulling specific information from text or documents**.

Example Input Text:

“Dr. Sharma works in the Computer Science department and has 15 years of experience.”

Extracted Data:

Field	Value
Name	Dr. Sharma
Department	Computer Science
Experience	15 years

Applications:

- ❖ Resume parsing
- ❖ Research paper analysis
- ❖ Medical records processing

Data Extraction

Activity 1 — Extract Information from an Email

Real Use Case

Organizations receive many emails containing important details like **meeting schedules, requests, and deadlines**. AI tools extract this information to create tasks automatically.

Email assistants in tools like Microsoft Outlook and Gmail use similar techniques.

Extract the following details from the email:

Sender Name

Meeting Date

Meeting Time

Meeting Topic

Email:

Hello team, this is Dr. Reddy. We will have a meeting on March 25 at 10:00 AM to discuss the new AI curriculum updates.

Data Extraction

Activity 2 — Extract Important Details from a News Paragraph

Real Use Case

News monitoring systems extract **key entities** such as events, organizations, and locations from articles.

Media monitoring platforms like Meltwater perform this type of extraction.

Extract the following information from the news text:

Event

Location

Organization Involved

Text:

The Ministry of Education launched a new digital learning initiative in New Delhi to promote AI education in universities.

Data Extraction

Activity 3 — Extract Invoice Information

Real Use Case

Businesses automatically extract data from invoices for accounting systems.

Many finance systems use AI-powered invoice processing.

Extract the following information from the invoice description:

Invoice Number

Vendor Name

Total Amount

Invoice Date

Invoice Text:

Invoice #4582 issued by Tech Supplies Ltd on April 5, 2026 for office equipment totaling ₹18,500.

Data Extraction

Activity 4 — Extract Event Details from an Announcement

Real Use Case

Event management systems extract information from announcements to create event listings.

Platforms like Eventbrite process event details automatically.

Extract the following details from the announcement:

Event Name

Date

Venue

Speaker

Announcement:

A workshop titled "Introduction to Generative AI" will be held on July 10 at ABC Engineering College auditorium. The session will be delivered by Dr. Anita Sharma.

Output Validation

Ensuring that the generated output:

- ☐ Follows the **required schema**
- ☐ Contains **correct data types**
- ☐ Meets **expected constraints**

Example validation rules:

- Age must be a number
- Email must follow valid format
- Required fields must not be empty

Benefits:

- ☐ Prevents incorrect data
- ☐ Ensures system reliability
- ☐ Improves data quality

Output Validation

Activity 1 — Email Format Validation

Real Use Case

Online platforms validate email addresses during registration to ensure they follow the correct format.

Platforms like Gmail and LinkedIn check if an email looks valid before creating an account.

Check whether the following email addresses are valid.

Rules:

- Must contain "@"
- Must contain a domain name

Emails:

1. ravi.kumar@gmail.com
2. anita.sharma.com
3. student@university.edu

Output Validation

Activity 2 — Age Validation in Registration Forms

Real Use Case

Web forms validate age fields to ensure users meet minimum requirements.

Many online services enforce **age restrictions** during registration.

Validate the following ages for a system that requires users to be at least 18 years old.

Ages:

15

21

17

30

Output Validation

Activity 3 — Date Format Validation

Real Use Case

Booking systems ensure that dates follow a **standard format** before storing them.

Travel platforms such as MakeMyTrip validate dates for hotel or ticket bookings.

Validate whether the following dates follow the format DD-MM-YYYY.

Dates:

12-05-2026

2026/05/12

31-11-2025

ReAct Prompting

What is ReAct Prompting?

ReAct stands for **Reasoning + Acting**.

It is a prompting technique where AI models:

- ❑ **reason about a problem**
- ❑ **take actions using tools or external information**
- ❑ **observe results**
- ❑ **continue reasoning until a final answer is reached**

This allows AI systems to **solve complex problems step-by-step**.

Why ReAct Prompting is Important

Traditional AI responses:

- generate answers directly
- may hallucinate or guess

ReAct prompting helps AI:

- ❑ perform **step-by-step reasoning**
- ❑ interact with **external tools**
- ❑ **verify intermediate results**
- ❑ solve **complex multi-step tasks**

Applications include:

- research assistants
- question answering systems
- AI agents

Reason + Act Concept

ReAct combines **two capabilities of AI systems**:

Reason

The AI **thinks through the problem logically**.

Example:

Identify what information is needed to answer the question.

Act

The AI **performs an action** such as:

searching the web

using a calculator

querying a database

This combination enables **intelligent decision-making**.

Thought–Action–Observation Framework

ReAct uses a repeating cycle:

Thought → **Action** → **Observation**

Thought

AI reasons about the problem.

Action

AI performs an operation (tool call, search, calculation).

Observation

AI analyzes the result returned by the action.

The cycle continues until the **final answer is produced**.

Example of Thought–Action–Observation

Question:

"What is the population of the capital of Japan?"

Thought:

Find the capital of Japan.

Action:

Search: “capital of Japan”

Observation:

Result: Tokyo

Thought:

Find population of Tokyo.

Action:

Search: “Tokyo population”

Observation:

Result: Population information found.

Final Answer: Population of Tokyo.

Tool Usage in ReAct

ReAct enables AI to **use external tools** to complete tasks.

Common tools include:

- Web search tools
- Calculators
- Databases
- APIs
- Code execution environments

Using tools improves **accuracy and reliability** of AI responses.

Multi-Step Reasoning

Many real-world problems require **multiple reasoning steps**.

Examples:

- solving math problems
- analyzing research papers
- planning travel itineraries
- debugging code

ReAct allows the model to **break problems into smaller steps** and solve them sequentially.

ReAct Prompting

Activity 1 — Finding a Train Route Between Cities

Real Use Case

Travel assistants used by platforms like IRCTC or Google must **search schedules, check routes, and determine travel options**.

This requires **multi-step reasoning and tool usage**.

Use the ReAct reasoning style to answer the question.

Question: What is the best way to travel from Hyderabad to Bangalore by train?

Follow this format:

Thought:

Action:

Observation:

Final Answer:

ReAct Prompting

Activity 2 — Restaurant Recommendation Using Location

Real Use Case

Food delivery assistants such as Zomato and Swiggy must reason through **location, cuisine preference, and ratings**.

Answer using the ReAct pattern.

Question: Suggest a good restaurant for South Indian food near Hyderabad city center.

Use the structure:

Thought

Action

Observation

Final Answer

ReAct Prompting

Activity 3 — Budget Planning for a Trip

Real Use Case

Travel planning assistants must **calculate expenses step-by-step** using reasoning and tools.

Platforms like Airbnb provide similar planning assistance.

Solve this using ReAct reasoning.

Question: Estimate the total budget for a 2-day trip to Hyderabad if hotel cost is ₹3000 per night and daily food expense is ₹1000.

Use the format:

Thought

Action

Observation

Final Answer

ReAct Prompting

Activity 3 — Budget Planning for a Trip

Real Use Case

Travel planning assistants must **calculate expenses step-by-step** using reasoning and tools.

Platforms like Airbnb provide similar planning assistance.

Solve this using ReAct reasoning.

Question: Estimate the total budget for a 2-day trip to Hyderabad if hotel cost is ₹3000 per night and daily food expense is ₹1000.

Use the format:

Thought

Action

Observation

Final Answer



Let's stay connected, my phone
number +91-9922369797
For LinkedIn,
Scan this QR code!!

